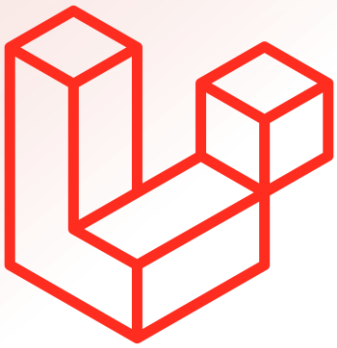


Laravel

The PHP Framework for Web Artisans

Note Index	Page Title	Topic
14	Seeder	Insert multi data External File
15	Query Builder	Show Data (Select)
16	Query Builder	Select Command-I
17	Query Builder	Select Command-II
18	Query Builder	Insert data
19	Query Builder	Update
20	Query Builder	Delete
21	Query Builder	Insert Data using form
22	Query Builder	Show & Update data using form
23	Query Builder	DELETE data using form
24	Query Builder	Pagination
25	Query Builder	Pagination Method
26	Query Builder	Pagination Method-II
27	Query Builder	Join Tables
28	Query Builder	Union & When
29	Query Builder	Chunk
30	Query Builder	Raw SQL
31	Validation	Form Validation
32	Validation	Custom Validation Rule
33	Validation	Form Request Validation
34	Validation	Form Request Validation-II
35	Validation	Form Custom Validation Rule
36	Controller	Resource Controller
37	Controller	Nested Resource Controller
38	Eloquent	Read Data : Get() Method's
39	Eloquent	Get() Method's -I
40	Eloquent	Create method (Insert data)
41	Eloquent	Update method (Update data)
42	Eloquent	Update method (Update data)-II
43	Eloquent	Delete method (Delete data)
44	Eloquent	Delete method (Delete data)-II
45	Eloquent	Advance CRUD Methods
46	Model	Model Conventions
47	Eloquent	One to One Relation Read: Data
48	Eloquent	One to One Relation Read: Method
49	Eloquent	One to One Relation Create: Data
50	Eloquent	One to One Relation Create: Data-II
51	Eloquent	One to One Invert (ulta) Relation
52	Eloquent	One to Many Relation Read: Data

Note Index	Page Title	Topic
53	Eloquent	One to Many Relation Read:Data-II
54	Eloquent	One to Many Relation Create: Data
55	Eloquent	One to Many Relation Create: Data-II
56	Eloquent	Many to Many Relation Read: Data
57	Eloquent	Many to Many Relation Create: Data
58	Eloquent	Has One Through Relationship
59	Eloquent	Has One Of Many Through Relation
60	Eloquent	Has One Of Many Through Relation-II
61	Eloquent	Has One of Many Relation Create: Data
62	Eloquent	Has Many Through Relation Read
63	Eloquent	Has Many Through Relation Read-II
64	Eloquent	One to One Polymorphic Relation: Read
65	Eloquent	One to One Polymorphic Relation: Read Create-II
66	Eloquent	One to One Polymorphic Relation: Read Create-IV
67	Eloquent	One to Many Polymorphic Relation
68	Eloquent	One to Many Polymorphic Relation: Create-II
69	Eloquent	One to Many Polymorphic Relation Read Create III
70	Eloquent	One Of Many Polymorphic Relation
71	Eloquent	One Of Many Polymorphic Relation: Read-II
72	Eloquent	One of Many Polymorphic Relation Read-III
73	Eloquent	One of Many Polymorphic Relation Table-IV
74	Eloquent	Many to Many Polymorphic Relation
75	Eloquent	Many to Many Polymorphic Relationship-II
76	Eloquent	Many to Many Polymorphic Relation Create-III
77	Eloquent	Many to Many Polymorphic Relation Table-IV
78	Eloquent	With JSON
79	Eloquent	With JSON: (Read)-II
80	Eloquent	With JSON: (Insert, Update, Delete)-III
81	Eloquent, Model	Model Event
82	Eloquent, Model	Model Event-II
83	Eloquent, Model	Observes (Model Events)
84	Eloquent, Model	Observes (Model Events)-II
85	Eloquent, Model	Observes (Model Events)-II
86	Eloquent, Model	Query Scope : Local
87	Eloquent, Model	Query Scope : Local-II
88	Eloquent, Model	Query Scope : Global
89	Eloquent, Model	Query Scope : Global-II
90	Eloquent, Model	Query Scope Global External file

Note Index	Page Title	Topic
91	Eloquent, Model	Query Scope Global External file-II
92	Eloquent, Model	Get Specific Colum Data
93	Eloquent, Model	Get Specific Colum Data-II
94	Eloquent, Model	Get Specific Colum Data-III
95	File	Upload, Delete, Update
96	File	Upload, Delete, Update-II
97	File	Upload, Delete, Update-III
98	File	Upload, Delete, Update-IV
99	Model	Accessors & Mutators
100	Model	Accessors & Mutators-II
101	Authentication	Register & Login (View Files)
102	Authentication	Register & Login –II
103	Authentication	Register & Login -III
104	Middleware	With Authentication (View Files)
105	Middleware	With Authentication (middleware Files)-II
106	Middleware	With Authentication (route & model)-III
107	Middleware	With Authentication (Controller File)-IV
108	Session	Store, Show, Delete
109	Session	Store, Show, Delete-II
110	Gates	(View Files)
111	Gates	(Gate & route)-II
112	Gates	(Controller)-III
113	Gates	(Controller)-IV
114	Policy	(View Files)
115	Policy	(View Files & Route)-II
116	Policy	(User Controller & User Model)-III
117	Policy	(Policy & Migration)-IV
118	Policy	(Book Controller)-V
119	Send Email	(.env, route, view, contoller)
120	Send Email	(custom mailable class)-II
121	Send Email	Width Attachment
122	Send Email	Width Attachment-II
123	Send Email	Width Attachment-III
124	API Sanctum	(Route & Model)
125	API Sanctum	(AuthController: signup, logout)
126	API Sanctum	(AuthController: login)
127	API Sanctum	(PostController: index, store)
128	API Sanctum	(PostController: show, update)
129	Postman	Postman Parameter (Api Testing)

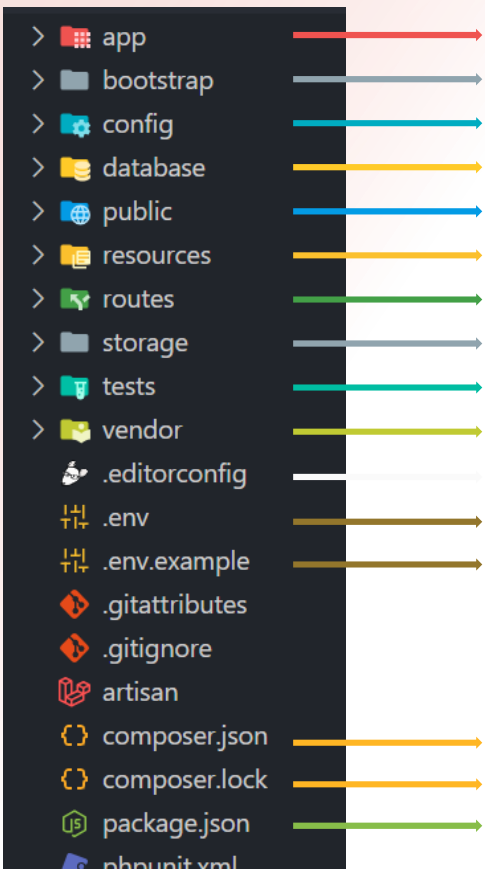
Laravel-11 : Create New Project, Folder Structure

0.1

Create Project : Using Laravel Installer

1	composer global require laravel/installer	(Install the Laravel Installer globally via Composer)
2	laravel new project-name	Create a new Laravel project
3	cd project-name	Serve the application
4	php artisan serve	Serve the application

Folder Structure in : Laravel-11



- app — Contains core application logic like Controllers, Models, Providers, etc.
- bootstrap — Contains bootstrap files for the application.
- config — Holds configuration files for various aspects of the application.
- database — Contains database-related files like migrations, factories, and seeders.
- public — Serves as the root directory for web assets like CSS, JavaScript, and images.
- resources — Contains resources like views, CSS, and JavaScript files.
- routes — Defines application routes for handling requests.
- storage — Used for storing files generated by the application.
- tests — Contains unit and integration tests for the application.
- vendor — Stores third-party packages managed by Composer.
- .editorconfig — Stores third-party packages managed by Composer.
- .env — Environment variables for the application.
- .env.example — Example environment variables file.
- composer.json — Project's Composer configuration file.
- composer.lock — File containing locked dependencies for the project.
- package.json — Configuration file for front-end dependencies.

Steps to Install : Composer, Laravel, PHP, and Set Up Localhost (XAMPP)

1. Install XAMPP (Localhost Server)
Download XAMPP from https://www.apachefriends.org/, install it, and start Apache and MySQL from the control panel. Verify installation by visiting http://localhost in your browser.

2. Install PHP
Download PHP from https://windows.php.net/ or use XAMPP's built-in PHP. Add PHP to your system's PATH environment variable. Verify the installation by running php --version in your terminal.

3. Install Composer
Download Composer from https://getcomposer.org/ and install it. Ensure Composer is added to your system PATH. Verify by running composer --version in the terminal.

Blade Template : Syntax

0.2

Syntax in PHP	Syntax in Blade Tamplete
`echo "hello";`	`{{"hello"}}`
`$name = "Yahoo Baba";` `echo $name;`	`{{$name}}`
`echo "<h1>Hello World</h1>";`	`{{!! <h1>Hello</h1> !!}}`
`<?php` `?>`	`@php` `@endphp`
`//commnet`	`{{-- comment --}}`

`if(condition){`
`}elseif(condition){`
`}else{}`
	`@if ()`
	`@elseif()`
	`@else`
	`@endif`

`if(isset($variable)){` `} //statement run when variable set`	`@isset($variable)` `@endisset`
`if(empty($variable)){` `} /`	`@empty($variable)` `@endempty`
`for($x = 1, $x <=10; $x ++){` `}`	`@for ($i = 0; $i < $count; $i++)` `@endfor`
`foreach($array_variable as $value){` `}`	`@foreach ($array_variable as $value)` `@endforeach`

`while(condition){`
`$x ++;`
`}`
	`@while (condition)`
	`@endwhile`

Blade Template : Syntax-II


```
|-------------------------------------------------------------------------
| This function use to import other page, (include subview)
| You can send value to header page in 2nd parameter
|-------------------------------------------------------------------------
|  @include('page.header')
|
|  @include('page.header',['name'=>'Bahoo Baba'])
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| Include file only if exist
|-------------------------------------------------------------------------
|  @includeIf('view.name', ['some' => 'data'])
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| This function include file only when condition true
|-------------------------------------------------------------------------
|  @includeWhen($boolean, 'view.name', ['some' => 'data'])
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| This function include file if condition false
|-------------------------------------------------------------------------
|  @includeUnless($boolean, 'view.name', ['some' => 'data'])
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| Condition on contact table but show data of Student table
|-------------------------------------------------------------------------
|  var data = @json($php_variable);
|  var data = {{Js::from($php_variable); }}
---------------------------------------------------------------------------


|-------------------------------------------------------------------------
| It is used to inherit a parent layout. A child view declares which parent
| layout it is extending.
|-------------------------------------------------------------------------
|  @extends('layout')  <!-- Inherit from layout.blade.php -->
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| It is used in the parent layout to define placeholder sections. These
| sections are filled by the child views.
|-------------------------------------------------------------------------
|  @yield('content')  <!-- Placeholder for content -->
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| It is used in the child view to define what content should go into a
| specific section (created by @yield).
|-------------------------------------------------------------------------
|  @section('content')  <!-- Fill the 'content' section -->
|      <p>This is dynamic content for the child view.</p>
|  @endsection
|-------------------------------------------------------------------------
```

Blade Template : Template Inheritance

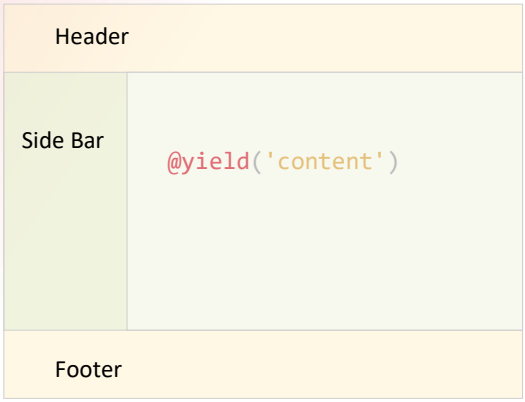
0.4

1 resources/views/layout.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <header>
        Header
    </header>

    <div class="sidebar">
        Side Bar
    </div>

    <div class="content">
        @yield('content')  <!-- Placeholder for dynamic content -->
    </div>

    <footer>
        Footer
    </footer>
</body>
</html>
```

2 resources/views/child.blade.php

```
@extends('layout')  <!-- Inherit from layout.blade.php -->

@section('content')  <!-- Provide content for the 'content' section -->
    <p>This is dynamic content from the child view.</p>
@endsection
```




```
@section('layout')
@section('content')
<p>this is dynamic</p>
@endsection
```

Template Inheritance in Laravel Blade Simplified:

Laravel's Blade template engine allows you to reuse layouts and create dynamic content using template inheritance. This is achieved through the following directives:
@extends(), @yield(), and @section().

How it Works:

The parent layout (layout.blade.php) defines the structure (e.g., header, sidebar, footer) and creates placeholders (@yield) for dynamic content.

The child view (child.blade.php) inherits this structure using @extends() and fills in the placeholders using @section()

Output:
When you load the child view in the browser, it will combine the layout from the parent and the dynamic content from the child

Summary:
@extends(): Links the child to the parent layout.
@yield(): Placeholder in the parent for dynamic content.
@section(): Content from the child view that goes into the placeholders.

Route View Route, Name Route, Redirect Route

1 Routes\web.php — Example of : View Rote

```php
<?php
use Illuminate\Support\Facades\Route;
Route::get('/', function () {
    return view('welcome');
}); //this example show how to create basic route in this route return view file
```

2 resources\views\welcome.blade.php — Example of : View Rote

```
<div>Home Page</div> // view file code
```

Create a home route flowing in web.php and call view file using view() method inside view() method give file name that you want call when user visit localhost:8000, this is home route

1 Routes\web.php — Example of : Name Rote

```php
<?php
use Illuminate\Support\Facades\Route;
Route::get('/', function () {
    return view('welcome');
})->name('welcome'); //name the route using name() method

--------------------------------------------------------------------------------
<a href="{{route('welcome')}}"></a> //use route name in blade file using route(name)
```

1 Routes\web.php — Example of : Redirect Rote

```php
<?php
use Illuminate\Support\Facades\Route;

Route::redirect('/welcome', '/maintenance', 301);
//when user visit at welcome page then user redirect to maintenance page
```

localhost:8000/

Home Page

Route Controller Route

1 **php artisan make: controller userController**

2 App\Http\Controllers\userController.php

```php
<?php
namespace App\Http\Controllers;
class userController extends Controller
{
    public function show() {
        return view('welcome'); //return view file in show method
    }
}
```

2 Routes\web.php

```php
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\userController;

Route::get('/',[userController::class,'show']);
//route call userController show method to return view file
```

2 resources\views\welcome.blade.php

```php
<h1>Home Page</h1> // view file code
```

If you want create a Controller follow 1 step run artian command then create show method in userController return view ‘welcome’ file , 3 step create Route in Route call show method

localhost:8000/

Home Page

Create Group Route

1 **php artisan make:controller userController**

2 App\Http\Controllers\userController.php

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class userController extends Controller {

public function home() { return view('welcome'); } //return welcome view file
public function about() { return view('about'); } //return about view file
public function user() { return view('user'); } //return user view file

}
```

3 Routes\web.php

```php
 <?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\userController;

Route::controller(userController::class)-> group( function () {
Route::get('/','home')->name('home'); //home route call home method
Route::get('/about','about')->name('about'); //about route call about method
Route::get('/user','user')->name('user'); //user route call user method
)};
```

4 resources\views\welcome.blade.php

```php
<h1>Home Page</h1>
```

5 resources\views\user.blade.php

```php
<h1>User Page</h1>
```

6 resources\views\about.blade.php

```php
<h1>About Page</h1>
```

Group route help to organize code, if you call same class multiple time and you have to call Controller::class to access class method, this way you call Controller::class only one time in side define Route in get method second parameter just define class method name

localhost:8000	localhost:8000/about	localhost:8000/user
Home Page	About Page	User Page

Create Table (Migration)

1 **php artisan make:migration create_students_table**

2 database\migrations\create_students_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint; //include Blueprint Class
use Illuminate\Support\Facades\Schema; //include Schema Class

return new class extends Migration
{

public function up(): void
    {
        Schema::create('student', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email');
            $table->intiger('phone');
            $table->string('city');
        }); //column names


public function down(): void
{
       Schema:: dropIfExists('student');
}

};
```

3 **php artisan migrate**

If you want to create table you have to create migration file step 1 command create migration file
Inside migration file under Schema create method declare column name, after run step 3 command
This will create table

Note : table name always end with “s” like ----> studens, ugers, customers, stocks,

Id	Name	Email	Phone	City (30)
1	Anjes Kumar	anjesh@email.com	8587878400	delhi
2	Ankit Kumar	ankit@email.com	8587878401	kolkata
3	Parshant Kumar	parshant@email.com	8587878402	chennai
4	Ashitosh Kumar	ashitosh@email.com	85878784203	mumbai

Create Migration Commands

1. **php artisan make:migration create_students_table**

This command use to create Table blue print or structure

2. **php artisan migrate**

This Command use to create Table according Blue Print

3. **php artisan migrate:rollback**

This command undo migration command or delete table

4. **php artisan migrate:rollback --step=4**

This command rollback or undo last 3 command

5. **php artisan migrate:reset**

This command delete all table, if your change anything in your migration file, after change run this command, after that run php artisan: migrate (2) command

5. **php artisan migrate: refresh**

This command delete all table and re create table, work like fist this command do rollback all migration, then run migrate command automatically

Id	Discription
`dd($json);`	Show data with extra information and below code not run (debug information)
`dump($json);`	Show data with extra infrormation

Column Modification

1 **php artisan make:migration update_students_table --table=students**

2 database\migrations\update_students_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('students', function (Blueprint $table) {
         $table->string('phone');  //add new column
         $table->dropColumn('persentage');  //delete Column
         $table->renameColumn('oldName','newName'); //rename column
         $table->string('city',30)->default('delhi')->change(); //change datatype
        });
    }

    public function down(): void
    {
        Schema::table('students', function (Blueprint $table) {
            //
        });
    }
};
```

3 **php artisan migrate**

If you want to change, update, or add column name or datatype you cant follow this stap under the table method define new column name or change datatype

Id	Name	Email	Phone	City (30)
1	Anjes Kumar	anjesh@email.com	8587878400	delhi
2	Ankit Kumar	ankit@email.com	8587878401	kolkata
3	Parshant Kumar	parshant@email.com	8587878402	chennai
4	Ashitosh Kumar	ashitosh@email.com	85878784203	mumbai

Column Datatype or Modifiers

Datatype & Modifiers in Laravle version 11

```php
$table->string('email')->nullable();
// for null value in column


$table->string('email')->unique();
// for unique value in column


$table->string('city')->default('Delhi');
// set default value in column


$table->primery('columnName');
// make column primary key


$table->foreign('user_id')->reference('id')->on('users');
// set user_id column as foreign key


$table->after('columnName');
//Place the column "after" another column


$table->primary('id')->autoincrement();
// Set Integer columns as auto-incrementing (primary key)


$table->first();
// Place the column "first" in the table


$table->from($integer_value);
// set the starting value of an auto-incrementing filed


$table->string('password')->invisible();
// make the column invisible to select command


$table->unsigned();
// set integer columns as unsigned


$table->useCuurent();
// set timestamp columns to use current timestamp as default value


$table->useCurrentOnUpdate()
// set timestamp columns to current timestamp when a record is updated
```

Here are all datatype and modifier in Laravel

Table Modification

1 **php artisan make:migration update_table**

2 database\migrations\update_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::rename('oldTableName','newTableName'); // rename Table
        Schema::dropIfExists('tableName'); // delete table
    }

    public function down(): void
    {
        //
    }
};
```

3 **php artisan migrate**

If you want to change, update, or add column name or datatype you cant follow this stap under the table method define new column name or change datatype

Id	Name	Email	Phone	City (30)
1	Anjes Kumar	anjesh@email.com	8587878400	delhi
2	Ankit Kumar	ankit@email.com	8587878401	kolkata
3	Parshant Kumar	parshant@email.com	8587878402	chennai
4	Ashitosh Kumar	ashitosh@email.com	85878784203	mumbai

Table Primary Key Foreign Key

1 **php artisan make:migration** create_students_table

2 database\migrations\create_students_table.php

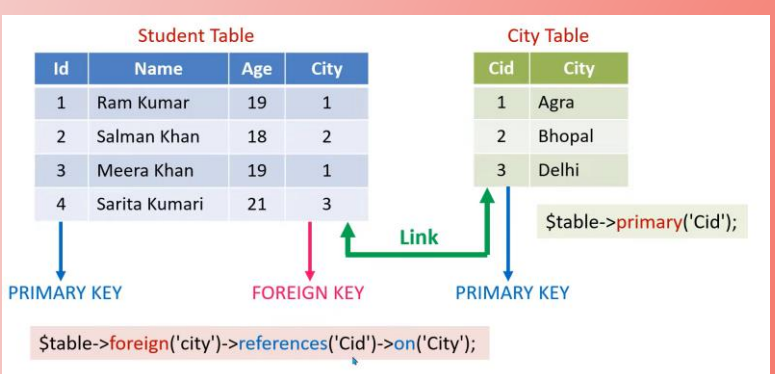
```php
public function up(): void
    {
        Schema::create('students', function (Blueprint $table) {
            $table->id();
            $table->string('name',30);
            $table->string('email',40)->nullable()->unique();

        });
    }
```

3 **php artisan make:migration** create_libraries_table

4 database\migrations\create_libraries_table.php

```php
public function up(): void
    {
        Schema::create('libraries', function (Blueprint $table) {
            $table->id();
            $table->unsignedBigInteger('student_id');
            $table->foreign('student_id')->references('id')->on('students');
            $table->string('book');
            $table->date('due_date');
            $table->boolean('status');
        });
    }
```

5 **php artisan** migrate



If your want to join 2 table column

Table Primary Key Foreign Key-II

1 **php artisan make:migration** create_students_table

2 database\migrations\create_students_table.php

```php
public function up(): void
    {
        Schema::create('students', function (Blueprint $table) {
            $table->id();
            $table->string('name',30);
            $table->string('email',40)->nullable()->unique();

        });
    }
```

3 **php artisan make:migration** create_libraries_table

4 database\migrations\create_libraries_table.php

```php
public function up(): void
    {
        Schema::create('libraries', function (Blueprint $table) {
            $table->id();

            $table->foreignId('student_id')->constrained('students')
                  ->cascadeOnUpdate()->cascadeOnDelete();

            $table->string('book');
            $table->date('due_date');
            $table->boolean('status');
        });
    }
```

5 **php artisan** migrate

This is second method to create table with relational table for make relation foregnId(‘pass pairent table name : like ’) foreginId(‘student_id’) note one thing remember pass table name without s For constrained(‘pass parents table name’) there are to new method use in this example One cascadeOnUpdate() or cascadeOnDelete() this both method run when parent table has on update or on delete i

Delete Primary or foreign Key

1 **php artisan make:migration** update_libraries_table --table=libraries

2 database\migrations\update_libraries_table.php

```php
public function up(): void
    {
        Schema::table('libraries', function (Blueprint $table) {
            $table->dropPrimary('column_name'); // remove primary key
            $table->dropUnique('column_name'); // remove unique constant
            $table->dropForeign(['student_id']); // remove foreign key

        });
    }
```

3 **php artisan** migrate:refresh

4 **php artisan** migrate

If you want to remove Primary key or Foreign Key follow above example
If you want to remover Primary Key only pass $table->dropPrimary('column_name');

Note: if you want to delete Primary Key or Foreign Key Column first you have to remove Primary or foreign constant

Seeder Insert Data

1 **php artisan make:model student**

2 **php artisan make:seeder StudentSeeder**

3 database\seeders\StudentSeeder

```php
<?php

namespace Database\Seeders;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use App\Models\student; // include model File

class StudentSeeder extends Seeder
{
    public function run(): void
    {
        //call model
        student::create([
            'name' => 'Anjesh Kumar',
            'email'=> 'anjesh@gmail.com'
        ]);
    }
}
```

4 database\seeders\DatabaseSeeder

```php
<?php
namespace Database\Seeders;
use App\Models\User;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            StudentSeeder::class //call seeder
        ]);
    }
}
```

5 **php artisan db:seed**

1 : Create Model model name & table name should be same (Table : students, Model : student)
2 : Create Seeder, seeder name must be start with database : Example : tabalenameSeeder
3 : include model file, and call model class in side run() method, (modelname::create([]))
4 : call seeder in DatabaseSeeder : example : $this->call([SeederName::class])

Seeder Insert multi data

1 **php artisan make:model student**

2 **php artisan make:seeder StudentSeeder**

3 database\seeders\StudentSeeder

```php
<?php
namespace Database\Seeders;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use App\Models\student;

class customeraddressSeeder extends Seeder {
    public function run(): void {
        $students = collect(
            [
                [
                    'name'=>'anush kumar',
                    'email'=> 'anush@gmail.com'
                ],
                [
                    'name'=>'anoj kumar',
                    'email'=> 'anoj@gmail.com'
                ],
                [
                    'name'=>'ocho kumar',
                    'email'=> 'abhiyans@gmail.com'
                ],
           ]);


        $students->each(function($student){
            customeraddress::create($student);
        });
```

4 database\seeders\DatabaseSeeder

```php
<?php
namespace Database\Seeders;
use App\Models\User;
use Illuminate\Database\Seeder;
class DatabaseSeeder extends Seeder {
    public function run(): void {
        $this->call([
            StudentSeeder::class //call seeder
        ]);
    }
}
```

5 **php artisan db:seed**

You can insert multiple data use any loop in this example we use create() method,
create() method prove loast of sub metho like each(), 1st you have to create collect() in side collect method create a array call each method on array like above example

Seeder Insert multi data External File

1 **php artisan make:model student**

2 **php artisan make:seeder StudentSeeder**

3 database\seeders\StudentSeeder

```php
<?php

namespace Database\Seeders;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\File; // include File Class
use App\Models\student; // include model file

class StudentSeeder extends Seeder
{
    public function run(): void
    {
        $json = File::get(path: 'database/json/students.json'); // json path
        $students = collect(json_decode($json)); // decode json to array

        $students->each(function ($student) {
             //call model class

            student::create([
                'name' => $student->name,
                'email' => $student->email,
            ]);
        });
    }
}
```

4 database\seeders\DatabaseSeeder

```php
<?php
namespace Database\Seeders;
use App\Models\User;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder {

    public function run(): void {

        $this->call([
            StudentSeeder::class //call seeder
        ]);
    }
}
```

5 **php artisan db:seed**

You can insert multiple data use any loop in this example we use create() method,
create() method prove loast of sub metho like each(), 1st you have to create collect() in side collect method create a array call each method on array like above example

Query Builder Show Data (Select)

1 **php artisan make:controller UserController**

2 App\Http\Controller\UserController.php

```php
<?php

namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; // include DB class

class UserController extends Controller
{
    public function showuser()
    {
        $users = DB::table(“students")->get(); //students is a table name with “s”

        return $users;
    }

    public function singleuser(string $id)
    {
        $users = DB::table(“students") ->where(‘id', $id)->get(); //get user id from
route
        return $users;
    }

}
```

3 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController;
use Illuminate\Support\Facades\Route;

Route::get('/', [UserController::class, 'showuser']);
Route::get('/student/{id}', [UserController::class, 'singleStudent']);

//call UserControler showuser method in routes
```

E Example DB Class use in if condition

```php
    public function showuser()
    {
       if(DB::table('orders')->where('id',1)->exists(){
            //statemante
        }

       if(DB::table('orders')->where('id',1)->doesntExist(){

        //statemante

       }
```

Query Builder Select Command-I

Query builder Method

```php
//this is example of where method : where(column,search_value)
$users = DB::table("students")->where('city','delhi')->get();

//this is example of select command
$users = DB::table("students")->select(name, email as User_email)
                              ->where('city', 'delhi')->get();

//this is example of multiple where method : where(columen,oprater,serch_value)
$users = DB::table("students")->select(name, email as User_email)
                              ->where('city', 'delhi')
                              ->where('age', '>', 20)->get();

//this is example of like exprestion : where(column,like,partten)
$users = DB::table("students")->where('name', 'like', 'S%')->get(); //name start
with S

//this is example of multiple condition in where method
$users = DB::table("students")->where([
                                    ['city', '=', 'delhi'],
                                    ['age', '>', 20],
                                ])->get();

//this is example of where and orwhere return data if both or one condition match
$users = DB::table("students")->where('city','=', 'delhi')
                              ->orWhere('age','>', 20)->get();


//this is example of where Between return date between range
$users = DB::table("students")->->whereBetween('age',[18,20])->get();


//this exapmpple where and orwherebetween return data if both or one condition match
$users = DB::table("students")->where('city','=', 'delhi')
                              ->orWhereBetween('age',[18,20])->get();

//this is example of where not in return all data wher city name not = mumbat,delhis
$users = DB::table("students")->->whereNotIn('city',['delhi','mumbai'])->get();


//this is example of where null return all data where row null
$users = DB::table("students")->->whereNull('email')->get();
```

Query Builder Select Command-II

Query builder Method

```php
//this example of where Date return all data where date = 2024-05-29
$users = DB::table("students")->whereDate('created_at','2024-05-29')->get();


//this example of where Month return all data where month = 5th (May)
$users = DB::table("students")->whereMonth('created_at','5')->get();


//this example of where Day, return all data where day = 29
$users = DB::table("students")->whereDay('created_at','29')->get();


//this example of where Year, return all data where year = 2024
$users = DB::table("students")->whereYear('created_at','2024')->get();


//this example of where Time, return all data where time = 08-01-32
$users = DB::table("students")->whereTime('created_at','08:01:32')->get();


//this example of orderBy this arrange data in to ascending order A-Z
$users = DB::table("students")->orderBy('name')->get();


//this example of orderBy this arrange data in to decending order Z-A
$users = DB::table("students")->orderBy('name','desc')->get();


$users = DB::table("students")->find() //return first recode in table
$users = DB::table("students")->letest() //return latest recode in table
$users = DB::table("students")->oldest() //return oldest recode in table
$users = DB::table("students")->inRendomOrder() //return random recode in table
$users = DB::table("students")->count() //return total recode in table
$users = DB::table("students")->max('age') //return max age in age column
$users = DB::table("students")->min('age') //return min age in age column
$users = DB::table("students")->sum('due') //return total value in due column



//example of LIMIT or offset return 10 recode after 10 row recode use in pagination)
$users = DB::table("students")->->limit(10)->offset(10)->get();


//example of take or skip return 10 recode after 10 row recode (use in pagination)
$users = DB::table("students")->->take(10)->skip(10)->get();
```

Query Builder Insert data

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; //include DB Class

class UserController extends Controller
{
    public function addUser()
    {
        $user = DB::table('customeraddresses') //table name (customeraddresses)
                        ->insert([
                            'name' => 'RAJU SINGH',      //table column names
                            'email' => 'rajusingh@gmail',
                            'created_at' => now(),
                            'updated_at' => now(), //now() method add current time
                          ]);
    }
} //if you don't want to use created_at updated_at columns first make : model file
In side model file : public $timestaps = false;
```

```php
$user = DB::table('customeraddresses') //table name (customeraddresses)

            ->insertOrIgnore([
                            'name' => 'RAJU SINGH', //table column names
                            'email' => 'rajusingh@gmail', //unique columns name
                           ]);

//this method ignore or not add value if unique column value already exist



$user = DB::table('customeraddresses') //table name (customeraddresses)

            ->insertGetId([
                            'name' => 'RAJU SINGH', //table column names
                            'email' => 'rajusingh@gmail', //unique columns name
                           ]);

            //this method add data and return Id, this method work only if your
            table have autoincriment column
```

3 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/update', [UserController::class, 'updateUser']);
//call addUser Controller on update routes
```

Query Builder update

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; //include DB Class

class UserController extends Controller
{
    public function updateUser() //update method
    {
        $user = DB::table('customeraddresses') //table name(customeraddresses with s
            ->where('id', '=', 5) //condition where update data
            ->update([
                'name' => 'Abhiyansh Kumar',
                'email' => 'abhiyansh@email.com',
            ]);
    }

}                           //this example show how to update data
```

```php
//this increment method increce the value like : if age = 60, then age after add 65
$user = DB::table('customeraddresses')
             ->where('id', 14)          // condition where update value
             ->increment('age', 5);     //increment 5 in age column

//this decrement method minus 5 in age column like if age 65 then after minus age 60
$user = DB::table('customeraddresses')
             ->where('id', 14)          // condition where update value
             ->increment('age', 5);     //increment 5 in age column


//increment and decrement in intiger column and update other column in 3rd parameter
$user = DB::table('customeraddresses')
            ->where('id', 14)
            ->decrement('age', 5, ['name' => 'Tushar Kumar']);


//this method use for multiple column data increment or decrement
$user = DB::table('customeraddresses')
            ->where('id', 14)
            ->incrementEach([           //incrementEach decrementEach
                'age' => 5, //column name you want to encrement or decrement
                'id' => 4,
            ]);
```

3 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; // include controller file
use Illuminate\Support\Facades\Route;

Route::get('/update', [UserController::class, 'updateUser']);
//call updateUser Controller on update routes
```

Query Builder delete

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\DB; //include DB Class

class UserController extends Controller
{
    public function deleteUser() //delete method
    {
        $user = DB::table('customeraddresses') //table name(customeraddresses with s
            ->where('id',5) //condition where delete the data
            ->delete();
    }

}                           //this example show how you delete data in table
```

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\DB; //include DB Class

class UserController extends Controller
{
    public function deleteUser() //delete method
    {
        $user = DB::table('customeraddresses') //table name(customeraddresses with s
              ->truncate(); // this method delete table data and reset id column
    }

}

//this example show how you how delete table data using truncate method this method
delete all table data and reset id column 1
```

3 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

//call deleteUser Controller on delete routes
Route::get('/delete', [UserController::class, 'deleteUser']);
```

Query Builder Insert Data using form

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; //include DB Class
use Illuminate\Http\Request; //include Request Class

class UserController extends Controller
{
    public function addUser(Request $request)
    {
        $user = DB::table('user_tables') //table name with “s” user_tables
            ->insert([
                'name' => $request->name,
                'email' => $request->email,
                'city' => $request->city,
                'age' => $request->age,
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        if ($user) {
             // return redirect()->route('home');
                return '<h1>Data Save Successfully Added.</h1>';
        } else {
            return '<h1>Data Not Save</h1>';
        }
    }
```

3 resources\views\adduser.blade.php

```php
<div>
        <form action="{{ route('addUser') //add user route}}" method="POST">
            @csrf //user csrf token
            <input type="text" name="name" placeholder="name"/>
            <input type="text" name="email" placeholder="email"/>
            <input type="text" name="age" placeholder="age"/>
            <input type="text" name="city" placeholder="city"/>
            <input type="submit">
        </form>
    </div>
```

4 Routes\web.php

```php
<?php

use App\Http\Controllers\UserController; //include Controller file
use Illuminate\Support\Facades\Route;

//this route call addUser method in Usercontroller File route name use in form
Route::post('/adduser', [UserController::class, 'addUser'])->name('addUser');
Route::view('newuser', '/adduser'); //route for show form page
```

Query Builder Show & Update data using form

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; //include DB Class
use Illuminate\Http\Request; //include Request Class

class UserController extends Controller
{
    public function updateUser(Request $request, $id) //$id is provide by route
    {
        $user = DB::table('user_tables')
            ->where('id', $id) //pass user $id in where condition
            ->update([
                'name' => $request->name, //request is where all form data is store
                'email' => $request->email,
                'city' => $request->city,
                'age' => $request->age,
                'created_at' => now(),
                'updated_at' => now(),
            ]);
    }

    public function updatePage(string $id) //this method show user data in page
    {
        $user = DB::table('user_tables')->find($id); //find method use to get user
        return view('updateuser', ['data' => $user]); //return user data to view
    }
}
```

3 resources\views\updateuser.blade.php

```php
<div>        //pass user id in route method in second parameter
        <form action="{{route('update.user',$data->id)}}" method="POST">
            @csrf //csrf token is requre
            <input type="text" value="{{$data->name}}" name="name"/>
            <input type="text" value="{{$data->email}}" name="email" />
            <input type="text" value="{{$data->age}}" name="age"/>
            <input type="text" value="{{$data->city}}" name="city"/>
            <button type="submit">Update User</button>
        </form>
</div> //when form submit all field data send to UserController updateUser method
```

4 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;
//this route need user id and send to usercontroller file for show data in view file
Route::get('/updatepage/{id}', [UserController::class, 'updatePage']);
//this route get user id and send to UserController for update user data
Route::post('/updateuser/{id}', [UserController::class, 'updateUser'])
    ->name('update.user');
```

Query Builder DELETE data using form

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; //include DB Class
use Illuminate\Http\Request; //include Request Class

class UserController extends Controller
{
    public function getuser(string $id) //getting user id or other details
    {
        $user = DB::table('user_tables')->find($id); //find method for get user data
        return view('deleteuser', ['data' => $user]); //return user data to view

    }
    public function deleteUser($id) //create method for delete user
    {
        $user = DB::table('user_tables')->where('id', $id)->delete(); //delete user

     }

}
```

3 resources\views\deleteuser.blade.php

```blade
<div>
        <form action="" method=""> //this form for view user details
            @csrf
            <input type="text" value="{{$data->name}}" name="name"/>
            <input type="text" value="{{$data->email}}" name="email" />
            <input type="text" value="{{$data->age}}" name="age"/>
            <input type="text" value="{{$data->city}}" name="city"/>
            <button type="submit">Update User</button>
        </form>
              //this form redire delete.user route with id
        <form action="{{route('delete.user',$data->id)}}" method="POST">
            @csrf
            @method('DELETE')
            <button type="submit">Delete User</button>
        </form>
    </div>
```

4 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include UserController
use Illuminate\Support\Facades\Route;

Route::get('/user/{id}', [UserController::class, 'getuser']); //call getuser method
Route::post('/deleteuser/{id}', [UserController::class, 'deleteUser'])-
>name('delete.user');
//this route call delteUser method in UserController file
```

Query Builder Pagination

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB; //include DB Class
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function showUser()
    {
        //this example show how to make pagination and return to view file
        $users = DB::table("user_tables")->paginate(4);
        return view('showuser', ['data' => $users]);

        //this show paginate method extra parameters
        $users = DB::table("user_tables")->paginate(4, ['name','email'], 'pagename')
        return $users;
        // ->appends(['key' => 'value']); this method help to pass extra data in url
        // ->simplePaginate(4); this is other paginate method
        //->cursorPaginate(); this is other Paginate method with fast speed

    }
}
```

3 resources\views\deleteuser.blade.php

```php
<table><tr><th>Name</th><th>Email</th><th>Age</th><th>City</th><th>User Id</th></tr>
            @foreach ($data as $id => $user) // foreach loop to display user data
                <tr>
                    <td>{{ $user->name }}</td>
                    <td>{{ $user->email }}</td>
                    <td>{{ $user->age }}</td>
                    <td>{{ $user->city }}</td>
                    <td>{{ $user->id }}</td>
                </tr>
            @endforeach
        </table>

        //link method display pagination links in view file
        <div> {{ $data->links('pagination::bootstrap-5') }}</div>
        <div>
             Total Page : {{ $data->total()}} </br> // total method show total data
             Current Page : {{$data->currentPage()}} //show current page number
        </div>
```

4 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get("/showuser", [UserController::class, "showUser"]);
//call showUser method in UserController file
```

Query Builder Pagination Method

Method	Description
$paginator->count()	Get the number of items for the current page.
$paginator->currentPage()	Get the current page number.
$paginator->firstItem()	Get the result number of the first item in the results.
$paginator->getOptions()	Get the paginator options.
$paginator->getUrlRange($start, $end)	Create a range of pagination URLs.
$paginator->hasPages()	Determine if there are enough items to split into multiple pages.
$paginator->hasMorePages()	Determine if there are more items in the data store.
$paginator->items()	Get the items for the current page.
$paginator->lastItem()	Get the result number of the last item in the results.
$paginator->lastPage()	Get the page number of the last available page. (Not available when using simplePaginate).
$paginator->nextPageUrl()	Get the URL for the next page.
$paginator->onFirstPage()	Determine if the paginator is on the first page.
$paginator->perPage()	The number of items to be shown per page.
$paginator->previousPageUrl()	Get the URL for the previous page.
$paginator->total()	Determine the total number of matching items in the data store. (Not available when using simplePaginate).
$paginator->url($page)	Get the URL for a given page number.
$paginator->getPageName()	Get the query string variable used to store the page.
$paginator->setPageName($name)	Set the query string variable used to store the page.
$paginator->through($callback)	Transform each item using a callback.

Query Builder Pagination Method-II

Method	Description
$paginator->count()	Get the number of items for the current page.
$paginator->cursor()	Get the current cursor instance.
$paginator->getOptions()	Get the paginator options.
$paginator->hasPages()	Determine if there are enough items to split into multiple pages.
$paginator->hasMorePages()	Determine if there are more items in the data store.
$paginator->getCursorName()	Get the query string variable used to store the cursor.
$paginator->items()	Get the items for the current page.
$paginator->nextCursor()	Get the cursor instance for the next set of items.
$paginator->nextPageUrl()	Get the URL for the next page.
$paginator->onFirstPage()	Determine if the paginator is on the first page.
$paginator->onLastPage()	Determine if the paginator is on the last page.
$paginator->perPage()	The number of items to be shown per page.
$paginator->previousCursor()	Get the cursor instance for the previous set of items.
$paginator->previousPageUrl()	Get the URL for the previous page.
$paginator->setCursorName()	Set the query string variable used to store the cursor.
$paginator->url($cursor)	Get the URL for a given cursor instance.

Query Builder Join Tables

1 **php artisan make:controller StudentsController**

2 App\Http\Controllers\StudentsController

```php
<?php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //include DB class

class StudentController extends Controller
{
    public function showStudent()
    {
        $students = DB::table('students')
            ->join('cities', 'students.city', '=', 'cities.id') //join table
            ->select('students.*', 'cities.city_name') //selects columns
            ->where('cities.city_name', '=', 'Delhi') //where condition
            ->get();

        return $students; //return data
    }
} // for this example you need two table one name student & second cities add
initial data make seeder, you can use (LEFT JOIN, CROSS JOIN, INNER JOIN, RIGH JOIN)
```

3 Routes\web.php

```php
<?php
use App\Http\Controllers\Students; //include controller file
use Illuminate\Support\Facades\Route;

Route::get("/showstudent", [StudentsController::class, "showStudent"]);
//call showUser method in UserController file
```

If you want to join two Table data follow the steps first you have to make few files
migration file : students, cities
model file : student,citie
controller file

Query Builder Union & When

1 **php artisan make:controller StudentsController**

2 App\Http\Controllers\StudentsController

```php
<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //include DB Class

class StudentController extends Controller
{
    public function uniounData() //union function to get 2 table combined data
    {

        $lecturars = DB::table('lecturars') //first table
            ->select('name', 'email');

        $students = DB::table('students') //second table
            ->select('name', 'email')
            ->union($lecturars) //pass first table name in union method
            ->get();

        return $students; //this union method use to get 2 table combined data note
                          both table have same column data type & columns
    }



    public function whendata() // when method this function run when condition true
    {

        $studensts = DB::table('students')
            ->when(false, function ($query) {
                $query->where('age', '>', 22); //query run when condition true
            }, function ($query) {
                $query->where('age', '<', 22); //query run when condition false
            })->get();

        return $studensts;
    }
} //in when method first parameter only true or false value
```

3 Routes\web.php

```php
<?php

use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

//route for union & when method
Route::get("/uniondata", [StudentController::class, 'uniounData']);
Route::get("/whendata", [StudentController::class, 'whendata']);
```

Query Builder Chunk

1 **php artisan make:controller StudentsController**

2 App\Http\Controllers\StudentsController

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //include DB Class

class StudentController extends Controller
{
    public function chunkdata()
    {
        $students = DB::table('students')->orderBy('id') //orderBy mandatory
            ->chunk(3, function ($students) {
                foreach ($students as $student) {
                    echo $student->name . "<br>"; //statement for each data
                }
            });

    } //chunk method show all data but fetch data in chunk like if table have 10k
             data chunk show 10k data but fetch 100 data each time replace 3 to 100



    public function chunkUpdate()
    {
        $students = DB::table('students')->orderBy('id')
            ->chunkById(3, function ($students) {     //fetch data in chunk

               foreach ($students as $student) { //using each method to update data
                    DB::table('students')             //table
                        ->where('id', $student->id) //condition
                        ->update(['status' => true]); //new data
                }
            });

    } // if you have large number of data like 10k then use chunk method to DELETE
and UPDATE data in chunk this
```

3 Routes\web.php

```php
<?php

use App\Http\Controllers\StudentController; //include Controller File
use Illuminate\Support\Facades\Route;

Route::get("/chunkdata", [StudentController::class, 'chunkdata']);
Route::get("/chunkupdate", [StudentController::class, 'chunkUpdate']);
```

Query Builder Raw SQL

1 **php artisan make:controller StudentsController**

2 App\Http\Controllers\StudentsController

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; //include DB class

class StudentController extends Controller
{
    public function showdata()
    {
        $students = DB::select("SELECT * FROM students WHERE id = ?", [5]);
        return $students;

    } //ROW SELECT command

    public function insertdata()
    {
        $students = DB::insert("insert into students(name,email,age,city)
                values(?,?,?,?)", ['Anjesh Kumar', 'anjesh@email.com', 20, 5]);
        return $students;

    } //row INSERT command


    public function updatedata()
    {
        $students = DB::update("UPDATE students SET email = 'kumar@gmail.com' WHERE
id = ?", [21]);
        return $students;

    } // row UPDATE command

    public function deletedata()
    {
        $student = DB::delete("DELETE FROM students WHERE id = ?", [21]);
        return $student;
    }
} // row DELETE Command (thi all above example is how to run Raw SQL Command)
```

3 Routes\web.php

```php
<?php

use App\Http\Controllers\StudentController; //include Controller File
use Illuminate\Support\Facades\Route;

Route::get("/showdata", [StudentController::class, 'showdata']);
Route::get("/inserdata", [StudentController::class, 'insertdata']);
Route::get("/updatedata", [StudentController::class, 'updatedata']);
Route::get("/deletedata", [StudentController::class, 'deletedata']);
```

Form Validation

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;   //include Request Class

class UserController extends Controller
{
    public function addUser(Request $req) //use request class, make object $req
    {
        $req->validate([   //validation method
            'name' => 'required',                    //validation rule's
            'email' => 'required|email',
            'age' => 'required|numeric',
            'city' => 'required',
            'password' => 'required|alpha_num|min:6',
            'phone' => 'required|numeric|size:10',

        ], [
            'name.required' => 'bhai bina name ke kam nahi hoga' //validation errors
        ]); //this validation error show only this controller file

        return $req->all(); // all method of Request class to read all data
  }
} //more rule : https://laravel.com/docs/11.x/validation#available-validation-rules
```

3 resources\views\adduser.blade.php

```
//any() use to check in @error have any error all() use to show all error
@if ($errors->any()) <pre> @php print_r($errors->all()); @endphp </pre> @endif

 <form action="{{route('addUser')}}" method="POST">
       @csrf
       <input type="text" name="name" placeholder="name" value="{{old('name')}}"/>
       <span> @error('name') {{$message}} @enderror</span> //2nd way to show error
       <input type="text" name="email" placeholder="email"/>
       <input type="number" name="phone" placeholder="phone number"/>
       <input type="text" name="age" placeholder="age"/>
       <input type="text" name="password" placeholder="password"/>
       <input type="text" name="city" style="@error('city') border:1px solid red;
      @enderror"/> //3rd way to show error, you can use class attribute
       <input type="submit">

</form> //{{old('name')}} use to if page refresh input not empty values stay
```

4 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include Controller File
use Illuminate\Support\Facades\Route;

Route::view('/', 'adduser'); //route for home page using view
Route::post('/addUser', [UserController::class, 'addUser'])->name('addUser');
```

Custom Validation Rule

2 Vendor\laravel\framework\src\illuminate\Translation\lang\en\validation.php

```php
<?php

    |--------------------------------------------------------------------------
    | Custom Validation Language Lines
    |--------------------------------------------------------------------------

    'custom' => [
        'username' => [
            'required' => 'Name required',
        ],

        'useremail' => [
            'required' => 'Email is Required',
            'email' => 'Email should in valid email ',

        ],
    ],

    |--------------------------------------------------------------------------
    | Custom Validation Attributes
    |--------------------------------------------------------------------------

    'attributes' => [
        'username' => 'User Name',
        'useremail' => 'Email Address',
        'userphone' => 'User Phone',
        'userage' => 'User Age',
    ],

];
```

Form Request Validation

1. **php artisan make:controller UserController**

2. **php artisan make:request UserRequest**

3. App\Http\Request\UserRequest

```php
<?php
namespace App\Http\Requests;
use Illuminate\Foundation\Http\FormRequest; //include FormRequest Class

class UserRequest extends FormRequest
{
    public function authorize(): bool {
        return true; //make true
    }


    public function rules(): array {
        return [
            'name' => 'required',
            'email' => 'required|email',
            'age' => 'required|numeric',
            'city' => 'required',
            'password' => 'required|alpha_num|min:6',
            'phone' => 'required|numeric',
        ];
    } // define all validation rule in rules() method



    public function messages() {
        return [
            'name.required' => 'bhai bina name ke kam nahi hoga',
            'email.required' => 'Enter :attributes '
        ];
    } //define all error message of validation rules like error massage of require



    public function attributes() {
        return [
            'name' => 'User Name',
            'email' => 'User Email',
            'age' => 'User age',
            'city' => 'User City'
        ];
    } //change your attributes name, if your key: username make User Name



    protected function prepareForValidation(){
        $this->merge([
            'name' => strtoupper($this->name),
        ]);
    } //this method use to modify data after validation like change text case
}
```

Form Request Validation

4 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;

use App\Http\Requests\UserRequest; //include UserRequest Class

class UserController extends Controller
{
  public function addUser(UserRequest $request){

  return $request->all(); //return all data in $request
  return $request->only(['name', 'email']); //this method return selected filed
  return $request->except(['email', 'password']); //return none selected filed

  }
}
```

5 resources\views\adduser.blade.php

```
<div> //any() use to check in @error have any error all() use to show all error

      @if ($errors->any())
          <pre>
             @php  print_r($errors->all());
             @endphp </pre>
      @endif


<form action="{{route('addUser')}}" method="POST">
   @csrf
   <input type="text" name="name" placeholder="name" value="{{old('name')}}"/>
       <span>
            @error('name')
                {{$message}}
            @enderror
       </span> //2nd way to show error

   <input type="text" name="email" placeholder="email"/>
   <input type="number" name="phone" placeholder="phone number"/>
   <input type="text" name="age" placeholder="age"/>
   <input type="text" name="password" placeholder="password"/>
   <input type="text" name="city" style="@error('city') border:1px solid red;
@enderror"/> //3rd way to show error, you can use class attribute

   <input type="submit">
</form>
</div> //{{old('name')}} use to if page refresh input not empty values stay
```

6 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include Controller File
use Illuminate\Support\Facades\Route;

Route::view('/', 'adduser'); //route for home page using view
Route::post('/addUser', [UserController::class, 'addUser'])->name('addUser');
```

Form Custom Validation Rule

1 **php artisan make:rule Uppercase**

2 App\Rules\Uppercasephp

```php
<?php
namespace App\Rules;
use Closure;
use Illuminate\Contracts\Validation\ValidationRule;
class Uppercase implements ValidationRule {

   public function validate(string $attribute, mixed $value, Closure $fail): void {
        if (strtoupper($value) !== $value) {
            $fail('The :attribute must be upper case');

 }}} //in this function define your validation
```

3 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request; //include request class
use App\Rules\Uppercase; //include custom validation class

class UserController extends Controller {
    public function addUser(Request $request) {

        $validatedData = $request->validate([
            'name' => ['required', 'string', new Uppercase], //use custom validation
        ]);

        return $request->all();
    }
}
```

4 resources\views\adduser.blade.php

```blade
//any() use to check in @error have any error all() use to show all error
@if ($errors->any()) <pre> @php print_r($errors->all()); @endphp </pre> @endif

 <form action="{{route('addUser')}}" method="POST">
       @csrf
       <input type="text" name="name" placeholder="name" value="{{old('name')}}"/>
       <span> @error('name') {{$message}} @enderror</span> //2nd way to show error
       <input type="submit">

</form> //{{old('name')}} use to if page refresh input not empty values stay
```

5 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include Controller File
use Illuminate\Support\Facades\Route;

Route::view('/', 'adduser'); //route for home page using view
Route::post('/addUser', [UserController::class, 'addUser'])->name('addUser');
```

Controller Resource Controller

1 **php artisan make:controller UserController --resource**

2 App\Http\Controllers\UserController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request; //include request class

class UserController extends Controller {

    public function index() {
        echo 'All User Data.'; //http://localhost:8000/users
    }

    public function create() {
        echo 'Open Add New User Page'; //http://localhost:8000/users/create
    }

    public function store(Request $request) {
        echo 'Add Data by form.';
    }

    public function show(string $id) {
        echo 'Details Of User : ' . $id; //http://localhost:8000/users/2
    }

    public function edit(string $id) {
        echo 'Open Edit Page With User Id' . $id;
        //http://localhost:8000/users/2/edit
    }

    public function update(Request $request, string $id) {
        echo 'Update the user' . $id;
    }

    public function destroy(string $id) {
        echo 'Delete the user' . $id;
    }
}
```

3 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::resource('users', UserController::class); //routes for resource controller
//the only method allow routes only define in only method
Route::resource('users', UserController::class)->only(['create','show']);
//except method all route but not allow which is define in except method
Route::resource('users', UserController::class)->except(['update','show']);
/
Route::resource('users', UserController::class)->names([
    'create' => 'users.build','show' => 'users.view']); //rename the route name

//php artisan route:list --name=users //this command show all route related to user
```

Controller Nested Resource Controller

1 **php artisan make:controller CommentController --resource**

2 App\Http\Controllers\CommentController

```php
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request; //include request class

class CommentController extends Controller {

    public function index() {

        echo 'All User Comment.'; //users/{user}/comment
    }

    public function create() {

        echo 'Open Add New User Page'; // users/{user}/comment/create
    }

    public function store(Request $request) {

        echo 'Add Data by form.'; //users/{user}/comment
    }

    public function show(string $id) {

        echo 'Details Of User : ' . $id; // users/{user}/comment/{comment}
    }

    public function edit(string $id) {

        echo 'Open Edit Page With User Id' . $id;
        // // users/{user}/comment/{comment} edit
    }

    public function update(Request $request, string $id) {
        echo 'Update the user' . $id; // users/{user}/comment/{comment}
    }

    public function destroy(string $id) {
        echo 'Delete the user' . $id; // users/{user}/comment/{comment}
    }
}
```

3 Routes\web.php

```php
<?php

use App\Http\Controllers\UserController; //include controller file
use App\Http\Controllers\CommentController; //include controller file
use Illuminate\Support\Facades\Route;

Route::resource('users.comment', UserController::class)->shallow();

//php artisan route:list --name=comments //this command show all route related to
user
```

Eloquent Read Data : Get() Method's

1 **php artisan make:model Student –controller**

2 App\Models\Student.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model {
    use HasFactory;
    public $timestaps = false; //define global variable if you don't have created_at
}                  or updated_at column in your table
```

3 App\Http\Controllers\StudentController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Student;     //include above model file student
use Illuminate\Http\Request;

class StudentController extends Controller {

 public function show() {//define method

     $students = Student::all(); //show all data in table return json
                 return $students;
  }
}



if (Student::where('id', 1)->exists()) {
    echo 'Rcode Foud'; //statement when recode found
}

if (Student::where('id', 60)->doesntExist()) {
   echo 'Rcode Not Foud'; //statement when recode not found
}

//query debug method's in laravel
$students = Student::whereAge(23)
            ->select('name', '')
            ->toRawSql();
            ->dd();
            ->ddRawSql();
```

4 Routes\web.php

```php
<?php
use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/students', [StudentController::class, 'show']); //define route for show
```

Eloquent Get() Method's -I

3 App\Http\Controllers\StudentController.php

```php
$students = Student::all(); //show all data in table

$students = Student::find(5); //find primary key column return data in found

$students = Student::find([2, 4], ['name', 'email']);
//multiple find second parameter for show only name or email column data

$students = Student::min('age'); //return minimum value in column

$students = Student::max('age'); //return maximum value in column

$students = Student::sum('age'); //return total value in column

$students = Student::where('name', 'Rohit Mehta')->get(); //where condition

$students = Student::where('age', '>', 23)->get(); //second way of where condition

$students = Student::where([  //multiple where condition
                            ['city', 'Delhi'], //first condition
                            ['age', '>', 23] //second condition
                           ])->get();

$students = Student::where('city', 'Delhi')->orWhere('age', '>', 23)->get();
//where with orWhere if both or one conation true then return data

$students = Student::whereAge(23)->get(); //short hand of where condition write
Where then your column name in camel case like this whereAge(20) this method
Find 20 in age column

$students = Student::whereCity('Delhi')->get(); //second example shorthand where

$students = Student::whereAge(23)
            ->select('name', 'email')
            ->get(); //shorthand where method with select method for show only name
                     and email column data

$students = Student::whereAge(23)
            ->select('name as Student Name', 'email as User email')
            ->get(); //select method with alias name change column key name

$students = Student::whereAge(23)->first(); //return fist recode

$students = Student::whereNot('city', 'Delhi')->get();
//return all data but not return dose data which city = Delhi

$students = Student::whereBetween('age', [20, 23])->get();
//return all data of age column which have between 20 to 23 age range

$students = Student::whereIn('city', ['Delhi', 'Mumbai'])->get();
//return all data of city column which have city = Delhi & Mumbai

$students = Student::whereNotIn('city', ['Delhi', 'Mumbai'])->get();
//return all data of city column where city not = Delhi & Mumbai

join(), leftJoin(), rightJoin(), crossJoin(), Union(), when(), chunk() //Learn more
about those method in query Builder
```

Eloquent Create method (Insert data)

1 **php artisan make:model Student –controller**

2 App\Models\Student.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model {
    use HasFactory;
    public $timestamps = false; //make timestamps value false if your don’t need
    protected $guarded = []; //protect field if empty array field not protected
    protected $fillable = ['name', 'email', 'age', 'city'];
}
```

2 App\Http\Controllers\StudentController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Student; //include model file
use Illuminate\Http\Request; //include request class

class StudentController extends Controller {
    public function addStudent(Request $request){//define method to add student
        $students = Student::create([
            'name' => $request->name,
            'email' => $request->email,
            'age' => $request->age,
            'city' => $request->city
        ]);
     return redirect()->route('home')->with('status', 'Student Added Sucssefully');
 }} //redirect to home route with massage : with(key,massage)
```

3 resources\views\adduser.blade.php

```
//using session to check in if condition if any data exist then show using session
<div> @if (session('status')) {{session('status')}} @endif </div>

        <form action="{{route('addstudent')}}" method="POST"> //addstudent route
            @csrf
            <input type="text" name="name" placeholder="name"/>
            <input type="text" name="email" placeholder="email"/>
            <input type="text" name="age" placeholder="age"/>
            <input type="text" name="city"/>
            <input type="submit">
        </form>
```

4 Routes\web.php

```php
<?php
use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;
Route::view('/', 'addStudent')->name('home'); //home route
Route::post('/addstudent', [StudentController::class, 'addStudent'])-
>name('addstudent'); // route to call StudentController addStudent Class
```

Eloquent Update method (Update data)

1 **php artisan make:model Student –controller**

2 App\Models\Student.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model {

    use HasFactory;
    public $timestamps = false; //make timestamps value false if your don’t need
    protected $guarded = []; //protect field if empty array field not protected
    protected $fillable = ['name', 'email', 'age', 'city']; //allow to add data
}
```

3 App\Http\Controllers\StudentController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Student; //include model file
use Illuminate\Http\Request; //include Request class

class StudentController extends Controller
{
    public function showStudent()
    {
        $students = Student::all();
        return view('student', compact('students'));

    } //fetch all student data and return to view student.blade.ph

    public function editStudent(string $id)
    {
        $student = Student::find($id);

        return view('updateStudent', compact('student'));

    } //fetch student data by id and return to view updata.blade.php


    public function updateStudent(Request $request, string $id)
    {
        $student = Student::where('id', $id)
            ->update([
                'name' => $request->name,
                'email' => $request->email,
                'age' => $request->age,
                'city' => $request->city
            ]);

     return redirect()->route('home')->with('status', 'Student Update Sucssefully');
    } //update student data and redirect to home route with massage
}
```

Eloquent Update method (Update data)-II

4 resources\views\student.blade.php

```
<div> //checking massage getting from controller file using if condition
    @if (session('status'))
        {{session('status')}}
    @endif
</div>
<table> //create a table to show all student data
    <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Age</th>
        <th>City</th>
        <th>Delete</th>
    </tr>
    @foreach ($students as $student) //using earch loop to display student data
    <tr>
        <td>{{$student->name}}</td>
        <td>{{$student->email}}</td>
        <td>{{$student->age}}</td>
        <td>{{$student->city}}</td>
        <td><a href="{{route('editstudent',$student->id)}}"></a></td>
    </tr>
    @endforeach  </table>
```

5 resources\views\updateStudent.blade.php

```
<div>//checking massage getting from controller file using if condition
        @if (session('status'))
        {{session('status')}}
        @endif
</div>

 <form action="{{route('updatestudent',$student->id)}}" method="POST">
    @csrf
    @method('PUT') //user PUT method when you update data
    <input value="{{$student->name}}" type="text" name="name" />
    <input value="{{$student->email}}" type="text" name="email"/>
    <input value="{{$student->age}}" type="text" name="age" />
    <input value="{{$student->city}}" type="text" name="city"/>
    <input type="submit">
 </form> //show student data getting from editStudent controller
```

6 Routes\web.php

```
<?php
use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

//route for home call showStudent method
Route::get('/', [StudentController::class, 'showStudent'])->name('home');

Route::put('updatestudent/{id}', [StudentController::class, 'updateStudent'])-
>name('updatestudent'); //route for updatestudent call updatestudent method

Route::put('editstudent/{id}', [StudentController::class, 'editStudent'])-
>name('editstudent'); //route for editstudent call editStudent Method
```

Eloquent Delete method (Delete data)

1 **php artisan make:model Student –controller**

2 App\Models\Student.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model {

    use HasFactory;
    public $timestamps = false; //make timestamps value false if your don’t need
    protected $guarded = []; //protect field if empty array field not protected
    protected $fillable = ['name', 'email', 'age', 'city']; //allow to add data
}
```

3 App\Http\Controllers\StudentController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Student; //include model file
use Illuminate\Http\Request; //include request class

class StudentController extends Controller
{

    public function showStudent()
    {
        $students = Student::all();
        return view('student', compact('students'));

    } //method fetch all student data then return to view using compact method


    public function deleteStudent(string $id)
    {
        $student = Student::find($id); //first way to delete data common use method
        $student->delete();

        $student = Student::where('id', $id); //delete data using where condition
        $student->delete();

        $student = Student::whereId($id); //delete data using short where condition
        $student->delete();

        Student::destroy([12, 13]);//3rd way to delete data pass multiple id in array

        Student::truncate();//this method delete all table data and reset primarykey

    return redirect()->route('home')->with('status', 'Student Deleted Sucssefully');

    } //return massage to view using with method with key status
}
```

Eloquent Delete method (Delete data)-II

4 resources\views\student.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
</head>
<body>
    <div> //checking massage getting from controller file using if condition
        @if (session('status'))
            {{session('status')}}
        @endif
    </div>
    <table> //create a table to show all student data
        <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Age</th>
            <th>City</th>
            <th>Delete</th>
        </tr>
        @foreach ($students as $student) //using earch loop to display student data
        <tr>
            <td>{{$student->name}}</td>
            <td>{{$student->email}}</td>
            <td>{{$student->age}}</td>
            <td>{{$student->city}}</td>
            <td>
            <form action="{{route('deletestudent',$student->id)}}" method="POST">
                    @csrf
                    @method('DELETE')
                    <button type="submit">DELETE</button>
             </form> //when button click form submit and call deletestudent route
             </td>
        </tr>

        @endforeach
    </table>
</body>
</html>
```

5 Routes\web.php

```
<?php

use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

//route for home call showStudent data
Route::get('/', [StudentController::class, 'showStudent'])->name('home');

Route::delete('deletestudent/{id}', [StudentController::class, 'deleteStudent'])-
>name('deletestudent'); //route for deletestudent/id call deleteStudent method
```

Eloquent Advance CRUD Methods

1 **php artisan make:model Student –controller**

2 App\Http\Controllers\StudentController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Student;
use Illuminate\Http\Request;

class StudentController extends Controller {

    public function showStudent() {//this method find and create or update

        $students = Student::upsert(
            [
                'name' => 'Anushka Sharma',
                'email' => 'anusha@gmail.com',
                'age' => 50,
                'city' => 4
            ],
            ['email'],//column name for find column name shoude be unique
            ['name']//column name if recode found the updaate name column
        );
        return $students;


        $students = Student::updateOrCreate(
            ['name' => 'Anush kumar'], //column name for match data
            [
                'email' => 'anusha@gmail.com', //data to be update or create
                'age' => 50,
                'city' => 4
            ]
        ); return $students;
 //this method find if recode found then update the recode, if not found then update
the recode


        $students = Student::firstOrCreate(
            ['name' => 'Anusha kumari'], //column name for match data
            [
                'email' => 'anusha@gmail.com', //data to be update or create
                'age' => 20,
                'city' => 4
            ]
        ); return $students;
//this method return first recode if recode found otherwise create new recode


        Student::chunk(5, function ($students) {
            foreach ($students as $student) {
                echo $student->name . '<br>';
            }
        }); //this chunk method use for when you have large number of recode using
        chunk method you can read the data or update the recode or delete the data
    }
}
```

Eloquent Model Conventions

1 **php artisan make:model Student**

2 App\Models\Student.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    use HasFactory;

  protected $table = 'yahoo_baba_students';
  //if your table name different from your model name you can change your table name

  protected $primaryKey = 'student_id';
  //if your primary key column name not id then you have to define your primary key
  column name

  public $timestamps = false;
  //make value false if your table don't have created_at or updated_at columns

  const CREATED_AT = 'Created_on';
  const UPDATED_AT = 'Update_on';
  //if you want to change your created_at or updated_at Column name then define
  your new name here

  protected $attributes = [
        'city' => 'delhi',
    ];
  //if you want set default value of any column define here if user not proved
  city then default city save in table

  public $incrementing = false;
  protected $keyType = 'string';
  //if you change primary key integer to string make $incrementing variable value
  false, and keyType variable value string


  protected $dateFormat = 'U';
  //if you want to change create_at or updated_at time format then change u to
  your format


  protected $connection = 'sqlite';
  //if you don't want to use other database only for this model your can change
  connection value

  use HasUuids;  use HasUlids;


  protected $guarded = []; //guard the table column

  protected $fillable = ['name', 'email', 'age', 'city'];
  //if you allow column to update our create data your have to declare column name
}
```

Eloquent One to One Relation Read: Data

1. **php artisan make:model Student**

2. **php artisan make:model Contact**

3. **php artisan make:Controller StudentController**

Students Table

```
+----+-------------+-----+
| id | Name        | Age |
+----+-------------+-----+
| 1  | Ram Kumar   | 19  |
| 2  | Salman Khan | 18  |
| 3  | Meera Khan  | 19  |
| 4  | Sarita Kumari| 21 |
+----+-------------+-----+
```

Contacts Table

```
+----+------------+--------+-----------+
| id | Phone      | City   | Student_id|
+----+------------+--------+-----------+
| 1  | 9988441122 | Agra   | 1         |
| 2  | 8833554477 | Mumbai | 2         |
| 3  | 2233665544 | Delhi  | 3         |
| 4  | 1122334455 | Mumbai | 4         |
+----+------------+--------+-----------+
```

4. App\Models\Student.php

```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    use HasFactory;

    public function studentContact() //custom method
    {
        return $this->hasOne(Contact::class);
    }
}
//in this custom method we join the contact table with student table,when call this
 method in controller file to get student & contact both table data
```

5. App\Http\Controllers\StudentController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\Student; //include model file

class StudentController extends Controller
{
    public function show()
    {
      $students = Student::with('studentContact')->get();
      return $students; //show both table all data
    }
} //studentContact is method we create in student model file to join both table
```

Eloquent One to One Relation Read: Method


```
|--------------------------------------------------------------------------
| Show both table all data
|--------------------------------------------------------------------------
|  $students = Student::with('studentContact')->get();
|  return $students;
|
|--------------------------------------------------------------------------


|--------------------------------------------------------------------------
| Show both table data with condition using where method
|--------------------------------------------------------------------------
| $students = Student::with('studentContact')->where('gender', 'F')->get();
| return $students;
|--------------------------------------------------------------------------


|--------------------------------------------------------------------------
| Show data with condition on foreign key table
|--------------------------------------------------------------------------
| $students = Student::withWhereHas('studentContact', function ($Query) {
|             $Query->where('city', 'Delhi');
|         })->get();
|         return $students;
|--------------------------------------------------------------------------


|--------------------------------------------------------------------------
| Show both table conditional data
|--------------------------------------------------------------------------
| $students = Student::where('gender', 'F')
|                     ->withWhereHas('studentContact', function ($Query) {
|                      $Query->where('city', 'Delhi');
|             })->get();
|             return $students;
|--------------------------------------------------------------------------


|--------------------------------------------------------------------------
| Condition on contact table but show data of Student table
|--------------------------------------------------------------------------
| $students = Student::where('gender', 'F')
|             ->whereHas('studentContact', function ($Query) {
|                 $Query->where('city', 'Delhi');
|             })->get();
|         return $students;
|--------------------------------------------------------------------------
```

6 Routes\web.php

```php
<?php
use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/', [StudentController::class, 'show']); //route for show student
```

Eloquent One to One Relation Create: Data

1. **php artisan make:model Student**

2. **php artisan make:model Contact**

3. **php artisan make:Controller StudentController**

4. App\Models\Student.php

```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    use HasFactory;
    public $timestamps = false; //if you don't use create_at or update_at column
    protected $guarded = []; //empty array allow all column to fill data

    public function studentContact() //custom method
    {
        return $this->hasOne(Contact::class);
    }
}
//in this custom method we join the contact table with student table,when call this
 method in controller file to get student & contact both table data
```

5. App\Models\Contact.php

```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Contact extends Model
{
    use HasFactory;
    public $timestamps = false; //if you don't use create_at or update_at column
    protected $guarded = []; //empty array allow all column to fill data
}
```

6. Routes\web.php

```php
<?php
use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/add', [StudentController::class, 'addStudent']);
//route for add student
```

Eloquent One to One Relation Create: Data-II

7 App\Http\Controllers\StudentController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Student; //include both model file
use App\Models\Contact;

use Illuminate\Http\Request;

class StudentController extends Controller
{

    public function addStudent()
    {
        $student = Student::create([
            'name' => 'Priyanka Kumari',
            'age' => 17,
            'gender' => 'F',
        ]); //insert data in Student table


        $student->studentContact()->create([
            'email' => 'priyanka@email.com',
            'phone' => '9953689473',
            'address' => 'Rosera Panchupur',
            'city' => 'Rosera',
        ]);

     }
}
```

insert data in contact table call method that we create in Student model (to join both table) now use same method to insert new data to both table in this way we don't have to use foreign key column this method automatically add primary key value to foreign key column

Students Table

id	Name	Age
1	Ram Kumar	19
2	Salman Khan	18
3	Meera Khan	19
4	Sarita Kumari	21

Contacts Table

id	Phone	City	Student_id
1	9988441122	Agra	1
2	8833554477	Mumbai	2
3	2233665544	Delhi	3
4	1122334455	Mumbai	4

Eloquent One to One Invert (ulta) Relation

1. **php artisan make:model Student**

2. **php artisan make:model Contact**

3. **php artisan make:Controller StudentController**

4. App\Models\Contact.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Contact extends Model
{
    use HasFactory;

    public function student()
    {
        return $this->belongsTo(Student::class);

    } //in this method we revers join student table to contact it’s mean we join
}       primary key to foreign key table
```

5. App\Http\Controllers\StudentController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Student; //include both model file
use App\Models\Contact;

use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function show()
    {
        $contacts = Contact::with('student')->get();
        return $contacts;//get invert data fist contact table data then student data
    }
}
```

6. Routes\web.php

```php
<?php
use App\Http\Controllers\StudentController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/', [StudentController::class, 'show']); //route for show student
```

Eloquent One to Many Relation Read: Data

1. **php artisan make:model User**

2. **php artisan make:model Post**

3. **php artisan make:Controller UserController**

Students Table

```
+----+-------------+-----+
| id | Name        | Age |
+----+-------------+-----+
| 1  | Ram Kumar   | 19  |
| 2  | Salman Khan | 18  |
| 3  | Meera Khan  | 19  |
| 4  | Sarita Kumari| 21 |
+----+-------------+-----+
```

Books Table

```
+----+------+------------+-----------+
| id | Book | Date       | Student_id|
+----+------+------------+-----------+
| 1  | ABC  | 01-01-2024 | 1         |
| 2  | XYZ  | 01-01-2024 | 1         |
| 3  | MNC  | 02-01-2024 | 3         |
| 4  | SRT  | 03-01-2024 | 3         |
+----+------+------------+-----------+
```

4. App\Models\User.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model {
    use HasFactory;

    public function posts() {
        return $this->hasMany(Post::class);
  }} //in this method we join Post table to User table (Primary Key to foreign Key)
```

5. App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\User;
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function show() {
        $user = User::with('post')->get();
        return $user;
    } //in this example show Post data using with method to show both table data
}
```

6. Routes\web.php

```php
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UserController; //include controller file

Route::get('/', [UserController::class, 'show']); //route for show method
```

Eloquent One to Many Relation Read:Data-II


```
|-------------------------------------------------------------------------
| Show both table all data
|-------------------------------------------------------------------------
| $user = User::with('post')->get();
| return $user;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show both table data using find method (show data by user id)
|-------------------------------------------------------------------------
| $user = User::with('post')->find(1);
| return $user;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show User who don't have any post (only show user data)
|-------------------------------------------------------------------------
| $user = User::doesntHave('post')->get();
| return $user;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show User who have post (only show user data if user belong to any post)
|-------------------------------------------------------------------------
| $user = User::has('post')->with('post')->get();
| return $user;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show User who have post (only show user data if user belong to any post)
|-------------------------------------------------------------------------
| $user = User::has('post')->get();
| return $user;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show User who have post (show with user data if user belong to any post)
|-------------------------------------------------------------------------
| $user = User::has('post')->with('post')->get();
| return $user;
|-------------------------------------------------------------------------
```

Students Table

id	Name	Age
1	Ram Kumar	19
2	Salman Khan	18
3	Meera Khan	19
4	Sarita Kumari	21

Books Table

id	Book	Date	Student_id
1	ABC	01-01-2024	1
2	XYZ	01-01-2024	1
3	MNC	02-01-2024	3
4	SRT	03-01-2024	3

Eloquent One to Many Relation Create: Data

1. **php artisan make:model User**

2. **php artisan make:model Post**

3. **php artisan make:Controller UserController**

Students Table

```
+----+-------------+-----+
| id | Name        | Age |
+----+-------------+-----+
| 1  | Ram Kumar   | 19  |
| 2  | Salman Khan | 18  |
| 3  | Meera Khan  | 19  |
| 4  | Sarita Kumari| 21 |
+----+-------------+-----+
```

Books Table

```
+----+------+------------+-----------+
| id | Book | Date       | Student_id|
+----+------+------------+-----------+
| 1  | ABC  | 01-01-2024 | 1         |
| 2  | XYZ  | 01-01-2024 | 1         |
| 3  | MNC  | 02-01-2024 | 3         |
| 4  | SRT  | 03-01-2024 | 3         |
+----+------+------------+-----------+
```

4. App\Models\User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;
    protected $guarded = []; //empty array allow to insert data in columns

    public function post()
    {
        return $this->hasMany(Post::class);
    }
} //in this method we join Post table to User table (Primary Key to foreign Key)
```

5. App\Models\Post.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $guarded = []; //empty array allow to insert data in columns
}
```

Eloquent One to Many Relation Create: Data-II

6 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Post; //include Post Model
use App\Models\User; //include user Model
use Illuminate\Http\Request;

class UserController extends Controller
{

    public function adduser()
    {
        $user = User::find(3);

        $user->post()->create([
            'title' => 'New Post Create by User 3',
            'description' => 'this is new post created by user 3'
        ]);

    } //in this method we find the user by id then call post method, we create in
        user model file using this method to joint both table then call create
       method to insert post data
}
|--------------------------------------------------------------------------
| Insert multiple Post data using create many method
|--------------------------------------------------------------------------

        $user = User::find(3);

        $user->post()->createMany(
            [
                [
                    'title' => 'New Post Create by User 3',
                    'description' => 'this is new post created by user 3'
                ],
                [
                    'title' => 'New Post Create by User 3',
                    'description' => 'this is new post created by user 3'
                ]
            ]
        );

//in this example we find user by id using find method, the insert multiple post
|--------------------------------------------------------------------------
| Not: one to many mean One ID have multiple recode on other table
|--------------------------------------------------------------------------
```

7 Routes\web.php

```php
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UserController; //include controller file

Route::get('/', [UserController::class, 'addUser']); //route for addUser method
```

Eloquent Many to Many Relation Read: Data

1. **php artisan make:model User**
2. **php artisan make:model Role**
3. **php artisan make:model user_role**
4. **php artisan make:Controller UserController**

5. App\Models\User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;

    public function roles() //custom method for join Roles table by user_roles table
    {
        return $this->belongsToMany(Role::class, 'user_roles');
    }
}
```

6. App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function show()
    {
        $user = User::find(5); //find user using find method
        return $user->roles; //return roles in side user
    }
}
```

7. Routes\web.php

```php
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UserController;

Route::get('/', [UserController::class, 'show']); //route for show method
Route::get('/adduser', [UserController::class, 'addUser']); //route for addUser
method
```

Eloquent Many to Many Relation Create:Data


```
|--------------------------------------------------------------------------
| Show user roles using find method in this example we find 5th user
|--------------------------------------------------------------------------
| $user = User::find(5);
| return $user->roles;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Show user roles using find method in this example we find 5th user
|--------------------------------------------------------------------------
|         $users = User::get();
|         foreach ($users as $user) {
|             echo $user->name . '<br>';
|             echo $user->eamil . '<br>';
|
|             foreach ($user->roles as $role) {
|                 echo $role->role_name . '<br>';
|             }
|
|             echo "<hr>";
|         }
|         $user = User::find(5);
|         foreach ($user->roles as $role) {
|             echo $role->role_name . '<br>';
|         }
|
|     }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| attach new role using attach method now user have 2 more role
|--------------------------------------------------------------------------
| $user = User::find(2);
| $user->roles()->attach([2, 3]);
| return $user->roles;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Delete user role you can pass role in detach method
|--------------------------------------------------------------------------
| $user = User::find(2);
| $user->roles()->detach(3);
| $user->roles()->detach(); // if method empty this delete all user role
| return $user->roles;
|--------------------------------------------------------------------------
```

id	name
1	Aarav
2	Vihaan
3	Vivaan
4	Ananya
5	Diya

id	role_name
1	editor
2	contributor
3	administrator
4	moderator
5	viewer

user_id	role_id
1	1
1	2
3	4
4	5
5	1

Eloquent Has One Through Relationship

1. php artisan make:model User
2. php artisan make:model Company
3. php artisan make:model Phone_number
4. php artisan make:Controller UserController

5. App\Models\User.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;
    public function CompanyNumber()
    {
        return $this->hasOneThrough(Number::class, Company::class);
    } //make relation between Number table and user table though company table

    public function company()
    {
        return $this->hasOne(Company::class);
    }
} //make relation between user table & Company table
```

Relation through

6. App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function show()
    {
        $users = User::with('company')->with('CompanyNumber')->get();
        return $users;
    }
} //use both relation we define in user.php (in model file) using with method
```

```
+----+---------+   +----+-------------+   +---------+-----------------+
| id | name    |   | id |  companies  |   | id      |    number       |
+----+---------+   +----+-------------+   +---------+-----------------+
|  1 | Aarav   |   |  1 | Google      |   |    1    |    444444444    |
|  2 | Vihaan  |   |  2 | Facebook    |   |    1    |    555555555    |
+----+---------+   +----+-------------+   +---------+-----------------+
```

Eloquent Has One Of Many Through Relation

1. **php artisan make:model Customer**
2. **php artisan make:model Order**
3. **php artisan make:controller CustomerController**
4. **php artisan make:Controller OrderController**

5. App\Models\Customer.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Customer extends Model
{
    use HasFactory;

    public function latestOrders() //custom name
    {
        return $this->hasOne(Order::class)->latestOfMany(); //show largest recode
    }
}
|--------------------------------------------------------------------------
| Show all recode (Order) related to customer
|--------------------------------------------------------------------------
|     public function orders() {
|         return $this->hasMany(Order::class);
|     }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Show all Oldest recode related to customer (order)
|--------------------------------------------------------------------------
|    public function oldestOrders() {
|        return $this->hasOne(Order::class)->oldestOfMany();
|    }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Show largest (max) recode in amount column of Order table
|--------------------------------------------------------------------------
|   public function largestOrders() {
|        return $this->hasOne(Order::class)->ofMany("amount", "max");
|    }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Show minimum (min) recode in amount column of Order table
|--------------------------------------------------------------------------
|    public function smallsetOrders() {
|        return $this->hasOne(Order::class)->ofMany("amount", "min");
|    }
|--------------------------------------------------------------------------
```

Eloquent Has One Of Many Through Relation-II

5 App\Http\Controllers\CustomerController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\Customer; //include Customer Model file
use Illuminate\Http\Request;

class CustomerController extends Controller
{
    public function show()
    {
        $customers = Customer::with('latestOrders')->get();
        return $customers;
    } //in this method we call latestorders method we define in Customer model file
}       to show all customer recode with latest order recode

|-------------------------------------------------------------------------
| Show customer recode with latest order related to customer in order table
|-------------------------------------------------------------------------
|       $customers = Customer::with('latestOrders')->find(2);
|        return $customers;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| show customer recode with largest order amount recode
|-------------------------------------------------------------------------
|        $customers = Customer::with('largestOrders')->find(2);
|        return $customers;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show all Customer data with order table recode related to the customer
|-------------------------------------------------------------------------
|       $customers = Customer::with('orders')->get();
|        return $customers;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show all Customer recode with orders table recode with latestOrder recode
|-------------------------------------------------------------------------
|       $customers = Customer::with('orders')->with('latestOrders')->get();
|         return $customers;
|-------------------------------------------------------------------------
```

Customer Table

id	name	email	phone
1	Yahoo	yahoobaba@email.com	99887744
2	Salman	salman@email.com	88447755
3	Deepika	deepika@email.com	77663322
4	Abhishek	abhishek@email.com	55668877

Order Table

id	amount	customer_id
1	250.00	2
2	3600.00	1
3	5400.00	3
4	5684.00	2

Eloquent Has One of Many Relation Create: Data

1. **php artisan make:model Customer**
2. **php artisan make:model Order**
3. **php artisan make:controller CustomerController**
4. **php artisan make:Controller OrderController**

5. App\Models\Order.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    use HasFactory;
    protected $guarded = []; //allow column to insert or update data
}
```

6. App\Http\Controllers\CustomerController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\Order; // include Order Model file
use Illuminate\Http\Request;

class OrderController extends Controller
{
    public function createOrder() //customer method
    {
        $order = Order::create([
            'amount' => 2500,
            'customer_id' => 2
        ]); //call create method to insert new data in order table
    }
}
```

Customer Table

id	name	email	phone
1	Yahoo	yahoobaba@email.com	99887744
2	Salman	salman@email.com	88447755
3	Deepika	deepika@email.com	77663322
4	Abhishek	abhishek@email.com	55668877

Order Table

id	amount	customer_id
1	250.00	2
2	3600.00	1
3	5400.00	3
4	5684.00	2

Eloquent Has Many Through Relation Read

1. **php artisan make:model Country**
2. **php artisan make:model User**
3. **php artisan make:model Post**
4. **php artisan make:Controller CountryController**

5. App\Models\Country.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Country extends Model
{
    use HasFactory;

    public function users() //custom method
    {
        return $this->hasMany(User::class); //join Country table with User table
    }

    public function posts() //custom method
    {
        return $this->hasManyThrough(Post::class, User::class);
    }
} //join Country table with Post table through User table
```

6. Routes\web.php

```php
<?php


use App\Http\Controllers\CountryController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/', [CountryController::class, 'show']); //route for show method
```

Relationship Explanation

The hasManyThrough method in the posts() relationship establishes the indirect connection between Country and Post.The first argument (Post::class) specifies the final related model (Post).The second argument (User::class) denotes the intermediate model linking Country and Post.

Eloquent Has Many Through Relation Read-II

7 App\Http\Controllers\CustomerController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Country; //include Model file
use Illuminate\Http\Request;

class CountryController extends Controller
{
    public function show() //custom method
    {
        $country = Country::find(2);
        return $country->posts; //return post of 2 id country

    }
}


|-------------------------------------------------------------------------
| Show Country data and Post data Related to Country 2
|-------------------------------------------------------------------------
|       $country = Country::with('posts')->find(2);
|         return $country;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show all Country data & with posts data (posts is a method in Country.php)
|-------------------------------------------------------------------------
|       $country = Country::with('posts')->get();
|           return $country;
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Show all Country with related user and posts
|-------------------------------------------------------------------------
|       $country = Country::with('users')->with('posts')->get();
|         return $country;
|-------------------------------------------------------------------------
```

Has Many through relationship tablel

Posts				Countries			Users	
id	Title	Detail	user_id	id	Name	id	Name	Country_id
1	Post One	News...	1	1	India	1	Amitabh	1
2	Post Two	News...	2	2	Pakistan	2	Salman	2
3	Post Three	News...	1	3	Bangladesh	3	Deepika	1
4	Post Four	News...	2			4	Shahid	3

Eloquent One to One Polymorphic Relation: Read

1. php artisan make:model Image
2. php artisan make:model User
3. php artisan make:model Post
4. php artisan make:Controller UserController
5. php artisan make:Controller PostController

6. App\Models\Image.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Image extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function imageable() //method name should be same of your column name
    {
        return $this->morphTo();
    }
} //Image table column name (id,imageable_id,imageable_type) use name for method
    before ‘id’, word
```

7. App\Models\Post.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function image() //custom method name
    {
        return $this->morphOne(Image::class, 'imageable');

    } //second argument is method name of Image Model file
} //join Post table to Image table using imageable() method of Image Model file
```

Eloquent One to One Polymorphic Relation: Read Create-II

8 App\Models\User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function image() //custom method name
    {
        return $this->morphOne(Image::class, 'imageable');
    }
} //join User table to Image table using imageable() method of Image Model file
```

9 App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\User; //include model file

class UserController
{
    public function show() //custom method to show user data with image table data
    {
        $user = User::with('image')->get();
        return $user;
    }

    public function addURL()
    {
        $user = User::find(3);
        $user->image()->create([
            'url' => 'user3.jpg'
        ]);
    } //custom method to insert 3 no user image data in image table
}
```

X Routes\web.php

```php
<?php
use App\Http\Controllers\PostController; //include controller file
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/', [UserController::class, 'show']); //route for show method
Route::get('/url', [UserController::class, 'addURL']); //route for add addURL method
Route::get('/post', [PostController::class, 'addPost']); //route for addPost method
```

11 App\Http\Controllers\PostController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Post; //include model file
use Illuminate\Http\Request;

class PostController
{
    public function addPost() //custom method
    {
        $post = Post::create([
            'title' => 'News title one',
            'description' => 'dsfsdhfosdfdsofjsdfodsf'
        ]);
        $post->image()->create([
            'url' => 'image/post-one.jpg'
        ]);
    }
} //custom method for insert new post data and new image data using image method we
    define in Post model file to join Post table to image table
```

12 TABLE Migration code

```
|-------------------------------------------------------------------------
| Create Post table (php artisan make:migration create_posts_table)
|-------------------------------------------------------------------------
| Schema::create('posts', function (Blueprint $table) {
|             $table->id();
|             $table->string('title');
|             $table->longText('description');
|             $table->timestamps();
|         });
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Create User table (php artisan make:migration create_users_table)
|-------------------------------------------------------------------------
| Schema::create('users', function (Blueprint $table) {
|             $table->id();
|             $table->string('name');
|         });
|
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Create Image table (php artisan make:migration create_images_table)
|-------------------------------------------------------------------------
| Schema::create('images', function (Blueprint $table) {
|             $table->id();
|             $table->string('url');
|             $table->unsignedBigInteger('imageable_id');
|             $table->string('imageable_type');
|         });
|-------------------------------------------------------------------------
```

Eloquent One to Many Polymorphic Relation

1. **php artisan make:model Comment**
2. **php artisan make:model Post**
3. **php artisan make:model Video**
4. **php artisan make:Controller VideoController**
5. **php artisan make:Controller PostController**

6. App\Models\Comment.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function commentable() //method name should be same of your column name
    {
        return $this->morphTo();
    } //Comment table column name (id,commentable_id,commentable_type) use name for
}       method before 'id', word
```

7. App\Models\Post.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');

    } //second argument is method name of Comment model file
} //join Post table to Comment table using comments() method of comment Model file
```

Eloquent One to Many Polymorphic Relation: Create-II

8 App\Models\Video.php

```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Video extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');

    } //second argument is method name of Comment model file
} //join Video table to Comment table using comments() method of comment Model file
```

9 App\Http\Controllers\PostController.php

```php
<?php

namespace App\Http\Controllers;
use App\Models\Post; //include Model file
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function addPost() //custom method for add new post with comment
    {
        $post = Post::create([
            'title' => 'New Post Title',
            'description' => 'New Post Discription‘

        ]); //add new post

        $post->comments()->create([          //add new comment call comment method
            'detail' => 'New Post Comment'   //define in post model file
        ]);
    }
}
```

X Routes\web.php

```php
<?php
use App\Http\Controllers\PostController; //include controller file
use App\Http\Controllers\VideoController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/', [VideoController::class, 'addConmment']); //route for addComment
Route::get('/show', [VideoController::class, 'show']); //route for show video data
Route::get('/post', [PostController::class, 'addPost']); //route for addPost Method
```

11 App\Http\Controllers\VideoController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Video; //include model file

class VideoController extends Controller
{
    public function show()
    {
        $video = Video::find(1);
        return $video->comments; //show Video table data with comment table data
    }
}
|--------------------------------------------------------------------------
| Show Video table data with Comment table data using foreach loop
|--------------------------------------------------------------------------
|    $video = Video::with('comments')->find(1);
|
|         echo "<h1>$video->title</h1>";
|         echo "<h4>$video->url</h4>";
|
|           foreach ($video->comments as $comment) {
|             echo $comment->detail;
|             echo '<hr>';
|         }
|--------------------------------------------------------------------------
| add new comment find the video id then add new comment using comment()
|--------------------------------------------------------------------------
|     public function addComment()  {
|         $video = Video::find(2);
|         $video->comments()->create([
|             'detail' => 'Good Videos'
|         ]);
|     }
|--------------------------------------------------------------------------
```

12 TABLE Migration code

```php
Schema::create('posts', function (Blueprint $table) {
            $table->id();
            $table->string('title');
            $table->longText('description');
            $table->timestamps();
        });
Schema::create('comments', function (Blueprint $table) {
            $table->id();
            $table->string('detail');
            $table->unsignedBigInteger('commentable_id');
            $table->string('commentable_type');
        });
 Schema::create('videos', function (Blueprint $table) {
            $table->id();
            $table->string('title');
            $table->string('url');
            $table->timestamps();
        });
```

Eloquent One Of Many Polymorphic Relation

1. **php artisan make:model Comment**
2. **php artisan make:model Post**
3. **php artisan make:model Video**
4. **php artisan make:Controller VideoController**
5. **php artisan make:Controller PostController**

6. App\Models\Comment.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function commentable() //method name should be same of your column name
    {
        return $this->morphTo();
    } //Comment table column name (id,commentable_id,commentable_type) use name for
}       method before 'id', word
```

7. App\Models\Post.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $guarded = []; //allow column to create or update data
    public $timestamps = false; //if create_at & update_at column not in your table

    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');

    } //second argument is method name of Comment model file
} //join Post table to Comment table using comments() method of comment Model file
```

8 App\Models\Video.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Video extends Model
{
    use HasFactory;
    protected $guarded = []; //allow column to create or update data

    public function latestComment()
    {
        return $this->morphOne(Comment::class, 'commentable')->latestOfMany();
    }
}   //in this method we join Video table to Comment table and get latest recode
      of Comment table, in morphOne() method 1st argument is comment table model
      class, 2nd argument is (commentable) is method in Comment model file

|-------------------------------------------------------------------------
| Get oldest recode of comment table using oldestOfMany()
|-------------------------------------------------------------------------
|     public function oldestComment()
|     {
|         return $this->morphOne(Comment::class, 'commentable')->oldestOfMany();
|     }
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Get maximum like column recode of comment table using
| OfMany(column name, aggregate function)
|-------------------------------------------------------------------------
|     public function bestComment()
|     {
|         return $this->morphOne(Comment::class, 'commentable')
|             ->OfMany("likes", "max");
|     }
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| get minimum like column data using ofMany(columnName, aggregate function)
|-------------------------------------------------------------------------
|     public function leastComment()
|     {
|         return $this->morphOne(Comment::class, 'commentable')
|             ->OfMany("likes", "min");
|     }
|-------------------------------------------------------------------------
```

9 Routes\web.php

```php
<?php
use App\Http\Controllers\VideoController;
use Illuminate\Support\Facades\Route;

Route::get('/show', [VideoController::class, 'show']); //route for show method
```

10 App\Http\Controllers\VideoController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Video;
use Illuminate\Http\Request;

class VideoController extends Controller
{
    public function show()
    {
        $video = Video::with('latestComment')->find(2);
        return $video;

    } //return 2nd id video data with latestComment this method define in Video Model
}       file

|-------------------------------------------------------------------------
| Show 2nd Video data with oldestComment this method define in Video model
|-------------------------------------------------------------------------
|       $video = Video::with('oldestComment')->find(2);
|       return $video;
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| Show 2nd Video data with bestComment this method define in Video model
|-------------------------------------------------------------------------
|       $video = Video::with('bestComment')->find(2);
|       return $video;
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| Show 2nd Video data with leastComment this method define in Video model
|-------------------------------------------------------------------------
|       $video = Video::with('leastComment')->find(2);
|       return $video;
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| Show 2nd Video only oldestComment data this method define in Video model
|-------------------------------------------------------------------------
|       $video = Video::find(2);
|       return $video->bestComment;
|-------------------------------------------------------------------------


|-------------------------------------------------------------------------
| Show 2nd Video only leastComment data this method define in Video model
|-------------------------------------------------------------------------
|       $video = Video::find(2);
|       return $video->leastComment;
|-------------------------------------------------------------------------
```

11 TABLE Migration code

```
|-------------------------------------------------------------------------
| Create Post table (php artisan make:migration create_posts_table)
|-------------------------------------------------------------------------
|       Schema::create('posts', function (Blueprint $table) {
|             $table->id();
|             $table->string('title');
|             $table->longText('description');
|             $table->timestamps();
|         });
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Create comments table (php artisan make:migration create_comments_table)
|-------------------------------------------------------------------------
|       Schema::create('comments', function (Blueprint $table) {
|             $table->id();
|             $table->string('detail');
|            $table->integer('detail');
|             $table->unsignedBigInteger('commentable_id');
|             $table->string('commentable_type');
|         });
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Create videos table (php artisan make:migration create_videos_table)
|-------------------------------------------------------------------------
|        Schema::create('videos', function (Blueprint $table) {
|             $table->id();
|             $table->string('title');
|             $table->string('url');
|             $table->timestamps();
|         });
|-------------------------------------------------------------------------
```

id	title	description	//Post Table
1	Post Title 1	Post Description 1	
2	Post Title 2	Post Description 2	
3	Post Title 3	Post Description 3	

id	title	url	created_at	//Video table
1	First Title	first video url	2024-07-28 12:00:00	
2	Second Title	second video url	2024-07-28 12:05:00	
3	Third Title	third video url	2024-07-28 12:10:00	

id	detail	commentable_id	commentable_type	created_at
1	Best Video	1	App\Models\Video	2024-07-28 12:00:00
2	This Video is Best	1	App\Models\Video	2024-07-28 12:05:00
3	Great Video	1	App\Models\Video	2024-07-28 12:10:00

Eloquent Many to Many Polymorphic Relation

1. php artisan make:model Tag --controller
2. php artisan make:model Post --controller
3. php artisan make:model Video --controller
4. php artisan make:model Taggables

5. App\Models\Tag.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Tag extends Model
{
    use HasFactory;
    protected $guarded = []; //allow column to create or update data

    public function posts()
    {
        return $this->morphedByMany(Post::class, 'taggable');

    } //custom method to join Tag table to Post table through taggable table

    public function videos()
    {
        return $this->morphedByMany(Video::class, 'taggable');

    } //custom method to join Tag table to Video table through taggable table
}
```

6. App\Models\Post.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;
    protected $guarded = []; //allow column to create or update data

    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');

    } //custom method to join Post table to Tag table through taggable table
}
```

Eloquent Many to Many Polymorphic Relationship-II

7 App\Models\Video.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Video extends Model
{
    use HasFactory;
    protected $guarded = []; //allow column to create or update data

    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');

    } //custom method to join Video table to Tag table through taggable table
}
```

8 App\Http\Controllers\TagController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Tag; //include model file
use Illuminate\Http\Request;

class TagController extends Controller
{
    public function show() //custom method
    {
        $tag = Tag::with('posts')->with('videos')->find(3);
        return $tag; //return all post and video data related to tag 3


        $tag = Tag::with(['posts:title', 'videos'])->find(3);
        return $tag; //short hand way of above method (with method),after coma pass
    }                column name example : table_name:name,age,city,phone
}
```

9 Routes\web.php

```php
<?php

use App\Http\Controllers\PostController; //include Post controller file
use App\Http\Controllers\TagController; //include  Tag controller file
use Illuminate\Support\Facades\Route;


Route::get('/post/create', [PostController::class, 'addPost']); //add post route
Route::get('/post/show', [PostController::class, 'show']); //show post route
Route::get('/tag/show', [TagController::class, 'show']); //show tag route
```

Eloquent Many toMany Polymorphic Relation Create-III

10 App\Http\Controllers\VideoController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Post; //include Model file
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function addPost()
    {
        $post = Post::create([
            "title" => "New Post Title 04",
            "description" => "this isa New post title 04 description"
        ]); //insert new post

        $post->tags()->create([
            "tag_name" => "#yahooBaba“

        ]); //insert new tag related to new post id
    }
|-------------------------------------------------------------------------------
| attach detach tag to post (link & unlink tag to post)
|-------------------------------------------------------------------------------
|         $post = Post::find(1);
|         $post->tags()->attach(3);
|
|         $post = Post::find(1);
|         $post->tags()->detach(3);
|-------------------------------------------------------------------------------

    public function show() //show tag related to post 3id
    {
        $post = Post::find(3);
        return $post->tags;

    }

    $post = Post::with('tags')->find(3); //show post and tag data related to post 3
    return $post;
}
```

Tags Table

id	Name
1	Bollywood
2	Hollywood
3	Salman Khan

Posts Table

id	Title
1	News Title One
2	News Title Two
3	New Title Three

Videos Table

id	Title	url
1	Video 1	you.be/xfd
2	Video 2	you.be/xfw

Taggables Table

tag_id	taggable_id	taggable_type
1	1	Post
3	1	Post
1	2	Video

11 TABLE Migration code

```
|-----------------------------------------------------------------------------
| Create Posts table (php artisan make:migration create_posts_table)
|-----------------------------------------------------------------------------
|     public function up(): void
|     {
|         Schema::create('posts', function (Blueprint $table) {
|             $table->id();
|             $table->string('title');
|             $table->longText('description');
|             $table->timestamps();
|         });
|
|     }
|-----------------------------------------------------------------------------

|-----------------------------------------------------------------------------
| Create Videos table (php artisan make:migration create_videos_table)
|-----------------------------------------------------------------------------
|     public function up(): void
|     {
|         Schema::create('videos', function (Blueprint $table) {
|             $table->id();
|             $table->string('title');
|             $table->string('url');
|             $table->timestamps();
|         });
|     }
|-----------------------------------------------------------------------------

|-----------------------------------------------------------------------------
| Create Tags table (php artisan make:migration create_tags_table)
|-----------------------------------------------------------------------------
|     public function up(): void
|     {
|         Schema::create('tags', function (Blueprint $table) {
|             $table->id();
|             $table->string('tag_name');
|             $table->timestamps();
|         });
|     }
|-----------------------------------------------------------------------------

|-----------------------------------------------------------------------------
| Create Taggable  table (php artisan make:migration create_taggables_table)
|-----------------------------------------------------------------------------
|     public function up(): void
|     {
|         Schema::create('taggables', function (Blueprint $table) {
|             $table->unsignedBigInteger('tag_id');
|             $table->unsignedBigInteger('taggable_id');
|             $table->string('taggable_type');
|
|         });
|     }
|-----------------------------------------------------------------------------
```

Eloquent With JSON

1 **php artisan make:model Test --controller**

2 **php artisan make:migration create_tests_table**

3 App\Models\Test.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Test extends Model
{
    use HasFactory;
    protected $guarded = []; //allow column to create or update data

    protected $casts = [
        "meta_data" => "json", //set column data type json like: column_name => json
    ];                         column data must be json if you store json data in column
}
```

4 Database\migrations\create_tests_table.php

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {

    public function up(): void
    {
        Schema::create('tests', function (Blueprint $table) {
            $table->id();
            $table->json("meta_data")->nullable(); //json column
            $table->timestamps();

        });//define your all column here
    }

    public function down(): void {
        Schema::dropIfExists('tests');
    }
};
```

5 Routes\web.php

```php
<?php
use App\Http\Controllers\TestController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/test/add', [TestController::class, 'saveData']);//route for save method
Route::get('/test/show', [TestController::class, 'show']); //route for show method
```

6 App\Http\Controllers\TestController.php

```php
<?php

namespace App\Http\Controllers;
use App\Models\Test; //include model file
use Illuminate\Http\Request;

class TestController extends Controller
{

    public function show() //custom method
    {
        $test = Test::find(1);
        return $test->meta_data; //return id 1 meta_data column recode
    }
}
|--------------------------------------------------------------------------
|  Show all data of Test table
|--------------------------------------------------------------------------
|  $test = Test::get();
|  return $test;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
|  Show data using where condition (column_name->key_name)
|--------------------------------------------------------------------------
|  $test = Test::where('meta_data->name', 'Anjesh Kumar')->get();
|  return $test;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
|  whereJsonContains special method of laravel (column_name->key_name)
|--------------------------------------------------------------------------
|  $test = Test::whereJsonContains('meta_data->name', 'Anjesh Kumar')->get();
|  return $test;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
|  (column_name->key_name) if name key have value then return data
|--------------------------------------------------------------------------
|  $test = Test::whereJsonLength('meta_data->name', 1)->get();
|  return $test;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
|  show only name key value of meta_data column (column_name->key_name)
|--------------------------------------------------------------------------
|  $test = Test::find(1);
|  return $test->meta_data['name']; //show only name key data
|   return $test->meta_data['address']['city']; //malty dimensional json
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
|  short json data by key name (column_name->key_name)
|--------------------------------------------------------------------------
|  $test = Test::orderBy('meta_data->name')->get();|  return $test;
|--------------------------------------------------------------------------
```

Eloquent With JSON: (Insert, Update, Delete)-III

6 App\Http\Controllers\TestController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Test; //include model file
use Illuminate\Http\Request;

class TestController extends Controller
{
    public function saveData() //custom method
    {

        $test = Test::create([
            "meta_data" => [
                "name" => "Anjesh Kumar",
                "email" => "anjesh@email.com",
                "phone_number" => "88776655",
            ]
        ]);

     } //insert new data in meta_data column of Test table
}
```

```
|-------------------------------------------------------------------------
|  UPDATE meta_data column recode, (column_name->key_name) using find
|-------------------------------------------------------------------------
|  $test = Test::find(1)->update([
|         "meta_data->name" => "Yahoo Baba",
|  ]);
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
|  UPDATE meta_data column recode, (column_name->key_name) using where
|-------------------------------------------------------------------------
|  $test = Test::where('id',1)->update([
|             "meta_data->name" => "Yahoo Baba",
|  ]);
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
|  DELETE meta_data column recode, meta_data is column, email is a key
|-------------------------------------------------------------------------
|  $test = Test::find(2);
|  $test->meta_data = collect($test->meta_data)->forget('email');
|  $test->save();
|-------------------------------------------------------------------------
```

Json Table

id	meta_data	created_at	updated_at
1	{"name":"Yahoo Baba","email":"baba@yahoo.com","phone_number":"22334455"}	2024-08-04	2024-08-04
2	{"name":"Anjesh Kumar","phone_number":"88776655"}	2024-08-04	2024-08-04

Eloquent Model Event

1 **php artisan make:model Post**

2 **php artisan make:model User --controller**

3 App\Models\User.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{    use HasFactory;

    public function posts()
    {
        return $this->hasMany(Post::class); //join User table to post table
    }

    protected static function booted(): void
    {
        static::deleted(function ($user) {//function run when delete query run
            $user->posts()->delete(); //delete post related to user when user delete
        });
}   }
|--------------------------------------------------------------------------
| Method automatic run after data created successfully of user table
|--------------------------------------------------------------------------
|       static::created(function ($user) {
|       });
|--------------------------------------------------------------------------
| Method automatic run before data creating of user table
|--------------------------------------------------------------------------
|       static::creating(function ($user) {
|       });
|--------------------------------------------------------------------------
| Method automatic run before data updating of user table
|--------------------------------------------------------------------------
|       static::updating(function ($user) {
|       });
|--------------------------------------------------------------------------
| Method automatic run after data update successfully of user table
|--------------------------------------------------------------------------
|       static::updated(function ($user) {
|       });
|--------------------------------------------------------------------------
| Method automatic run before data deleting of user table
|--------------------------------------------------------------------------
|       static::deleting(function ($user) {
|       });
|--------------------------------------------------------------------------
| Method automatic run before data read of user table
|--------------------------------------------------------------------------
|       static::retrieved(function ($user) {
|       });
|--------------------------------------------------------------------------
```

4 App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function deleteUser() //custom method for delete user table data
    {
        $user = User::find(2); //deleting user data, when this command run
        $user->delete();         automatically deleted (declare in model file) run
    }                            method, in delete method we delete post data
}                                related to user
```

5 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/user/delete', [UserController::class, 'deleteUser']); //route
```

6 TABLE Migration code

```php
Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email');
        });

Schema::create('posts', function (Blueprint $table) {
            $table->id();
            $table->string('title', 50);
            $table->string('slug', 100);
            $table->longText('description');
            $table->integer('counter')->default(0);
            $table->unsignedBigInteger('user_id');
            $table->foreign('user_id')->references('id')->on('users');
            $table->timestamps();
       });
```

```
+----+---------------+------------------+
| id | name          | email            |
+----+---------------+------------------+
| 1  | Alice Smith   | alice@email.com  |
| 2  | Bob Johnson   | bob@email.com    |
| 3  | Abhiyansh Kumar| kumar@email.com  |
| 4  | Sanchit Kumar | sanchit@gmail.com|
+----+---------------+------------------+
```

```
+----+---------------+----------------+---------------------------+---------+---------+
| id | title         | slug           | description               | counter | user_id |
+----+---------------+----------------+---------------------------+---------+---------+
| 1  | News Title One  | news-title-one   | Lorem ipsum dolor sit amet | 0       | 1       |
| 2  | News Title Two  | news-title-two   | Lorem ipsum dolor sit amet | 0       | 1       |
| 3  | News Title Three| news-title-three | Lorem ipsum dolor sit amet | 0       | 1       |
| 4  | News Title Four | news-title-four  | Lorem ipsum dolor sit amet | 0       | 1       |
+----+---------------+----------------+---------------------------+---------+---------+
```

Eloquent Observes (Model Events)

1 **php artisan make:model Post --controller**

2 **php artisan make:model observer PostObserver --model=Post**

3 App\Http\Controllers\PostController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Post; //include model file
use Illuminate\Http\Request;

class PostController extends Controller
{

    public function show()
    {
        $post = Post::find(10); //show post table data of 10th post
        return $post;
    }




    public function addPost()
    {
        $post = Post::create([
            'title' => '2nd Testing titel',
            'description' => '2nd Testing observes description',
            'user_id' => 3

        ]);
    }       //insert new post recode but here we don't declare slug column value
              we make slug in observer file, take title value and make slug
              automatic



                // Find a post by ID
             $post = Post::find(1);

             // Update the post's title without firing events
             $post->title = 'Yahoo Baba';
             $post->saveQuietly();

             // Delete the post without firing events
             $post->deleteQuietly();

             // Permanently delete the post without firing events
             $post->forceDeleteQuietly();

             // Restore a soft-deleted post without firing events
             $post->restoreQuietly();

}
```

Eloquent Observes (Model Events)-II

4 App\Providers\AppServiceProvider.php

```php
<?php

namespace App\Observers;

use App\Models\Post; //include model file
use Illuminate\Support\Str; //include Str class for

class PostObserver
{
    public function retrieved(Post $post): void //function run when post data read
    {
        $post->increment('counter'); //when data read incise counter column value
    }

    public function created(Post $post): void
    {
        //function run automatically when data save successfully
    }
    public function creating(Post $post): void //method run when data creating
    {
        $post->slug = Str::slug($post->title, "-"); //make slug using tile value
    }
    public function updated(Post $post): void
    {
        //method run when data updated successfully
    }
    public function deleted(Post $post): void
    {
        //method run when data deleted successfully
    }
    public function restored(Post $post): void
    {
        //method run when data restored
    }
    public function forceDeleted(Post $post): void
    {
        //method run when data deleted by force
    }
}
```

5 App\Observers\PostObserver.php

```php
<?php
namespace App\Providers;
use App\Models\Post; //include model file
use App\Observers\PostObserver; //include observer file
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Post::observe(PostObserver::class); //call above observer

    } //include all observer file related model file then call observer to run
}     //observer file automaticly
```

Eloquent Observes (Model Events)-II

5 Routes\web.php

```php
<?php

use App\Http\Controllers\PostController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/post/add', [PostController::class, 'addPost']); //route for add post
Route::get('/post/show', [PostController::class, 'show']); //route for show post
```

6 TABLE Migration code

```php
Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email');
        });

Schema::create('posts', function (Blueprint $table) {
            $table->id();
            $table->string('title', 50);
            $table->string('slug', 100);
            $table->longText('description');
            $table->integer('counter')->default(0);
            $table->unsignedBigInteger('user_id');
            $table->foreign('user_id')->references('id')->on('users');
            $table->timestamps();
       });
```

```
+----+---------------+------------------+
| id | name          | email            |
+----+---------------+------------------+
| 1  | Alice Smith   | alice@email.com  |
| 2  | Bob Johnson   | bob@email.com    |
| 3  | Abhiyansh Kumar| kumar@email.com  |
| 4  | Sanchit Kumar | sanchit@gmail.com|
+----+---------------+------------------+
```

```
+----+---------------+-----------------+----------------------------+---------+---------+
| id | title         | slug            | description                | counter | user_id |
+----+---------------+-----------------+----------------------------+---------+---------+
| 1  | News Title One  | news-title-one   | Lorem ipsum dolor sit amet  | 0       | 1       |
| 2  | News Title Two  | news-title-two   | Lorem ipsum dolor sit amet  | 0       | 1       |
| 3  | News Title Three| news-title-three | Lorem ipsum dolor sit amet  | 0       | 1       |
| 4  | News Title Four | news-title-four  | Lorem ipsum dolor sit amet  | 0       | 1       |
+----+---------------+-----------------+----------------------------+---------+---------+
```

Summary

•**Create** an observer class using php artisan make:observer.

•**Define** the event handling methods in the observer class.

•**Register** the observer in a service provider's boot method.

•**Trigger** the events by performing actions on the model.

Observers help to keep your code clean and maintainable by centralizing the event handling logic.

Eloquent Query Scope : Local

1 **php artisan make:model User --controller**

2 App\Models\User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;

       public function posts()
    {
        return $this->hasMany(Post::class); //join User Table To Post Table
    }


    public function scopeActiveUser($query) //method name activeUser()
    {
        return $query->where('status', 1); //where condition for status column
    }


    public function scopeCity($query, $cityName) //method name city()
    {
        return $query->whereIn('city', $cityName); //where condition for city comlun
    }                                               //here get city name dynamically

}
```

3 TABLE Migration code

id	name	email	city	status
1	Yahoo Baba	yahoobaba@email.com	Delhi	1
2	Salman Khan	salman@email.com	Mumbai	0
3	Deepika Padukone	deepika@email.com	Delhi	1
4	Abhishek Bachan	abhishek@email.com	Goa	1

id	title	description	status	user_id
1	News Title One	Lorem ipsum dolor sit amet, consectetur...	1	1
2	News Title Two	Lorem ipsum dolor sit amet, consectetur...	1	2
3	News Title Thre	Lorem ipsum dolor sit amet, consectetur...	1	1
4	News Title Four	Lorem ipsum dolor sit amet, consectetur...	1	3
5	News Title Five	Lorem ipsum dolor sit amet, consectetur...	1	4
6	News Title Six	Lorem ipsum dolor sit amet, consectetur...	0	1
7	News Title Seve	Lorem ipsum dolor sit amet, consectetur...	1	2

Eloquent Query Scope : Local-II

4 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function show()
    {
        $user = User::with('posts')->get(); //show all user with post table data



        $user = User::with('posts') //show all user with post table data
            ->city(['Goa', 'Delhi']) //use city method create in model file
            ->activeUser() //user activeUser method create in model file
            ->get();


        $user = User::select('id', 'name', 'email') //show only those column data
            ->with('posts:title,description,user_id') //show post specific column
            ->city(['Goa', 'Delhi'])//condition on city table method in create model
            ->activeUser() //method created in model file
            ->get();

        return $user;
    }
}
```

5 Routes\web.php

```php
<?php

use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/user/show', [UserController::class, 'show']); //route for show method
```

Defining a Local Scope

A local scope is defined as a method on the Eloquent model. The method name should start with scope, followed by the name you want to use to call the scope.

Using Local Scopes

Once a local scope is defined, you can call it as part of your query. The scope prefix is not included when calling the scope

Eloquent Query Scope : Global

1 **php artisan make:model User --controller**

2 App\Models\User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Builder; //include Builder Class
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;


    public function posts()
    {
        return $this->hasMany(Post::class); //join User Table to Post Table
    }


    protected static function booted(): void //default method name booted
    {

        static::addGlobalScope('userStatus', function (Builder $builder) {

            $builder->where('status', 1); //create query this query run automatic
                                          // for every User Model query
        });
    }
}
```

3 TABLE Migration code

id	name	email	city	status
1	Yahoo Baba	yahoobaba@email.com	Delhi	1
2	Salman Khan	salman@email.com	Mumbai	0
3	Deepika Padukone	deepika@email.com	Delhi	1
4	Abhishek Bachan	abhishek@email.com	Goa	1

id	title	description	status	user_id
1	News Title One	Lorem ipsum dolor sit amet, consectetur...	1	1
2	News Title Two	Lorem ipsum dolor sit amet, consectetur...	1	2
3	News Title Thre	Lorem ipsum dolor sit amet, consectetur...	1	1
4	News Title Four	Lorem ipsum dolor sit amet, consectetur...	1	3
5	News Title Five	Lorem ipsum dolor sit amet, consectetur...	1	4
6	News Title Six	Lorem ipsum dolor sit amet, consectetur...	0	1
7	News Title Seve	Lorem ipsum dolor sit amet, consectetur...	1	2

Eloquent Query Scope : Global-II

4 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function show()
    {
        $user = User::with('posts')->get(); //show all user with post table data

        $user = User::with('posts') //show all user with post table data

         // ->activeUser() /here we don't have to check user active or not
            ->get();


        $user = User::select('id', 'name', 'email') //show only those column data
            ->with('posts:title,description,user_id') //show post specific column

            // ->activeUser() //here we don't have to check user active or not
            ->get();          //our global scope query automatic check

        return $user;
    }
}
```

5 Routes\web.php

```php
<?php

use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/user/show', [UserController::class, 'show']); //route for show method
```

Defining a Local Scope

A local scope is defined as a method on the Eloquent model. The method name should start with scope, followed by the name you want to use to call the scope.

Using Local Scopes

Once a local scope is defined, you can call it as part of your query. The scope prefix is not included when calling the scope

Eloquent Query Scope Global External file

1 **php artisan make:model User --controller**

2 **php artisan make:scope UserScope**

3 App\Models\User.php

```php
<?php
namespace App\Models;
use App\Models\Scopes\UserScope; //include external scope file
use Illuminate\Database\Eloquent\Builder; //include builder class
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;

    public function posts()
    {
        return $this->hasMany(Post::class); //join User Table To Post table
    }


    protected static function booted(): void
    {
        static::addGlobalScope(new UserScope); //call external socope file
    }
}
```

4 App\Models\Scopes\UserScope.php

```php
<?php
namespace App\Models\Scopes;
use Illuminate\Database\Eloquent\Builder; //include builder calls
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Scope;

class UserScope implements Scope
{
    public function apply(Builder $builder, Model $model): void //default method
    {
        $builder->select('id', 'name', 'email')
            ->with('posts:title,description,user_id')
            ->where('status', 1);

    } //using $builder class add query, this query run automatically for every user
}       model query, in user query you don't have to use select,with,where, method
```

Global Query Scopes in Laravel:

•**Create a Scope Class**: Implement the apply method in a class that extends Illuminate\Database\Eloquent\Scope. This method defines the global query conditions.
•**Apply the Scope**: In your model, use the booted method to apply the scope using addGlobalScope.
•**Usage**: The scope is automatically applied to all queries on the model. To bypass it, use withoutGlobalScope.
•**Advantages**: Storing the scope in an external class promotes reusability, cleaner code, and consistent query logic across models.

Eloquent Query Scope Global External file-II

5 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User; //include model file
use Illuminate\Http\Request;
use App\Models\Scopes\UserScope; //include scope class

class UserController extends Controller
{
    public function show()
    {
        $user = User::get(); //her we don't user select,with,where method but inside
        return $user; //scope query automatically attach with this query, return all
                      //user data, where status =1,  with post table data

        $user = User::withoutGlobalScope(UserScope::class)->get();
        return $user; //withoutGloalScope this method use when you don't won't to
                        attach scope file query, in this query
    }
}
```

6 Routes\web.php

```php
<?php

use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/user/show', [UserController::class, 'show']); //route for show method
```

3 TABLE Migration code

```
+----+-----------------+----------------------+--------+--------+
| id | name            | email                | city   | status |
+----+-----------------+----------------------+--------+--------+
|  1 | Yahoo Baba      | yahoobaba@email.com  | Delhi  |      1 |
|  2 | Salman Khan     | salman@email.com     | Mumbai |      0 |
|  3 | Deepika Padukone| deepika@email.com    | Delhi  |      1 |
|  4 | Abhishek Bachan | abhishek@email.com   | Goa    |      1 |
+----+-----------------+----------------------+--------+--------+
+----+---------------+-------------------------------------------+-------+---------+
| id | title         | description                               |status | user_id |
+----+---------------+-------------------------------------------+-------+---------+
|  1 | News Title One | Lorem ipsum dolor sit amet, consectetur...  |     1 |       1 |
|  2 | News Title Two | Lorem ipsum dolor sit amet, consectetur...  |     1 |       2 |
|  3 | News Title Thre| Lorem ipsum dolor sit amet, consectetur...  |     1 |       1 |
|  4 | News Title Four| Lorem ipsum dolor sit amet, consectetur...  |     1 |       3 |
|  5 | News Title Five| Lorem ipsum dolor sit amet, consectetur...  |     1 |       4 |
|  6 | News Title Six | Lorem ipsum dolor sit amet, consectetur...  |     0 |       1 |
|  7 | News Title Seve| Lorem ipsum dolor sit amet, consectetur...  |     1 |       2 |
+----+---------------+-------------------------------------------+-------+---------+
```

Eloquent Get Specific Colum Data

1 **php artisan make:model User --controller**

2 **php artisan make:scope Post --controller**

3 App\Models\User.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    use HasFactory;

    protected $with = ['posts']; //this global variable attach post data of every
    User model query, you don't have to use with method in your query, posts is
    method name, method use to join user table to post table

    public function posts()
    {
        return $this->hasMany(Post::class); //1st way join User table to  post table
    }

    public function posts()
    {
      return $this->hasMany(Post::class)->select('title', 'description', 'user_id');
    }
} //2nd way join user table to post table with select method, this will show only
title description column data, note you must have declare foreign key column  name
```

4 App\Models\Post.php

```php
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    public function user()
    {
       return $this->belongsTo(User::class); //1st way join Post table to User table
    }

    public function user()
    {
        return $this->belongsTo(User::class)->select(['name', 'email', 'id']);
    }
} //2nd way join post table to user table with select method, this will show only
name email column data, note you must have declare primary key (id) column name
```

Eloquent Get Specific Colum Data-II

5 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function show()
    {
        $user = User::get();
        return $user;
    }
}
|--------------------------------------------------------------------------
| query return id 1 user : name email column data, find(id, column_name)
|--------------------------------------------------------------------------
|       $user = User::find(1, ["name", "email"]);
|--------------------------------------------------------------------------
| query return id 1 user : only email column data, find(id)->column_name
|--------------------------------------------------------------------------
|       $user = User::find(1)->email;
|--------------------------------------------------------------------------
| return all user name email column data get(column_name) all(column_name)
|--------------------------------------------------------------------------
|       $user = User::get(["name", "email"]);
|       $user = User::all("name", "email");
|--------------------------------------------------------------------------
| return user table data without post table data, if you use $with in model
|--------------------------------------------------------------------------
|       $user = User::without('posts')->get();
|--------------------------------------------------------------------------
| in your $with [] have multiple table name you don't want all table data
|--------------------------------------------------------------------------
|       $user = User::withOnly('posts')->get();
|--------------------------------------------------------------------------
| return fist recode only email column data
|--------------------------------------------------------------------------
|        $user = User::where('city', 'Delhi')->first()->email;
|--------------------------------------------------------------------------
| return fist recode only email column data (2nd way)
|--------------------------------------------------------------------------
|        $user = User::where('city', 'Delhi')->value("email");
|--------------------------------------------------------------------------
| accept only 2 parameter, return all data in one array pluck(key,value)
|--------------------------------------------------------------------------
|        $user = User::pluck("name", "email");
|--------------------------------------------------------------------------
| accept only 2 parameter, return all data in one array pluck(key,value)
|--------------------------------------------------------------------------
|        $user = User::all();
|        $user->load('posts');
|--------------------------------------------------------------------------
```

Eloquent Get Specific Colum Data-III

6 App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Post;
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function show()
    {
        $post = Post::with('user')->get();
        return $post; //return post table data with user table

        $post = Post::with('user')->get(['title', 'description', 'user_id']);
        return $post; //return post table data with user table specific column

        $post = Post::with('user:id,name,email')->get(['title',
'description','user_id']);
        return $post; //return post & user table specific column data

        $post = Post::with('user:id,name as User_name')->get(['title',
'description', 'user_id']);
        return $post; // return both table data, and rename column name (old as new)

        $post = Post::with('user')->get(['title', 'description', 'user_id']);
        return $post; //get user table all data but post table specific column data
    }
}
```

7 Routes\web.php

```php
<?php
use App\Http\Controllers\PostController; //include controller files
use App\Http\Controllers\UserController;
use Illuminate\Support\Facades\Route;
Route::get('/user/show', [UserController::class, 'show']); //route for user
Route::get('/post/show', [PostController::class, 'show']); //route for post
```

8 TABLE Migration code

```
+----+-----------------+----------------------+--------+--------+
| id | name            | email                | city   | status |
+----+-----------------+----------------------+--------+--------+
|  1 | Yahoo Baba      | yahoobaba@email.com  | Delhi  |      1 |
|  2 | Salman Khan     | salman@email.com     | Mumbai |      0 |
|  3 | Deepika Padukone| deepika@email.com    | Delhi  |      1 |
+----+-----------------+----------------------+--------+--------+

+----+---------------+-----------------------------------------------+-------+---------+
| id | title         | description                                   |status | user_id |
+----+---------------+-----------------------------------------------+-------+---------+
|  1 | News Title One| Lorem ipsum dolor sit amet, consectetur...    |     1 |       1 |
|  2 | News Title Two| Lorem ipsum dolor sit amet, consectetur...    |     1 |       2 |
|  3 | News Title Thre| Lorem ipsum dolor sit amet, consectetur...   |     1 |       1 |
|  4 | News Title Four| Lorem ipsum dolor sit amet, consectetur...   |     0 |       3 |
+----+---------------+-----------------------------------------------+-------+---------+
```

File Upload,Delete,Update

1 **php artisan make:model User --controller --resource**

2 **php artisan storage:link**

3 resources\views\upload-file.blade.php

```
<div>
        <h2>Upload File</h2>
            <form action="{{route('user.store')}}" method="POST"
                                                enctype="multipart/form-data">
            @csrf
            <input type="file" name="photo" accept=".png,.jpeg,.jpg">
            <input type="submit">
        </form> //form for upload file in action send data to store method
</div>

<div>
      @foreach ($users as $user)
         <div>
            <img src="{{asset('/storage/images/'.$user->file_name)}}"> //show image
              <form action="{{route('user.destroy',$user->id)}}" method="POST">
                  @method('DELETE')
                  @csrf
                  <button type="submit">DELETE</button>
                  <a href="{{route('user.edit',$user->id)}}">Update</a>
              </form> //here display user image with delete button & update page link
         </div>
      @endforeach
</div>
```

4 resources\views\update-file.blade.php

```
<div>
  <h2>Update File</h2> //form route send data to update method with user id
  <form action="{{route('user.update',$user->id)}}" method="POST"
        enctype="multipart/form-data"> //enctype multipart/form-data
        @csrf  @method('PUT') //define PUT method for update route

      //show old image of user get image from edit method from UserController file
       <img id="image" src="{{asset('/storage/images/'.$user->file_name)}}">

       <input type="file" name="photo" accept=".png,.jpeg,.jpg"
       onchange="document.querySelector('#image').src=window.URL.createObjectURL(this
       .files[0])" >// when user choose new image, hide old image then show new image

       <button type="submit">Update</button> //submit new image data to update route
       </form>
</div>
```

5 Routes\web.php

```
<?php
use App\Http\Controllers\UserController; //include controller file
use Illuminate\Support\Facades\Route;

Route::resource('user', UserController::class); //resource route for UserController
```

6 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;
use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index() //return all user data to view key name users
    {
        $users = User::get();
        return view('upload-file', compact('users'));
    }
|--------------------------------------------------------------------------
| Upload Image on server using 'storeAs' method in store method
|--------------------------------------------------------------------------
|   public function store(Request $request)
|   {
|       $file = $request->file('photo'); // file data in photo key
|       $fileName = time() . '_' . $file->hashName(); //generate new file name
|
|       $file->storeAs('images', $fileName, 'public'); //save in public/images
|       $file->storeAs(folder name, file name, public directory);
|
|       User::create([
|           'file_name' => $filename //add image path in database
|       ]);
|       return redirect()->route('user.index'); //redirect to index route
|   }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Get User data from database then send to update view file in key "user"
|--------------------------------------------------------------------------
|   public function edit(string $id)
|   {
|       $user = User::find($id); //fetch user data related to id (image path)
|       return view('update-file', compact('user')); //return data to view
|   }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| DELETE image on server when destroy route call
|--------------------------------------------------------------------------
|   public function destroy(string $id)
|   {
|         $user = User::find($id); //fetch user data related to $id
|         $user->delete(); //using delete method delete user data from database
|         $image_path = public_path('storage/images/') . $user->file_name;
|
|         if (file_exists($image_path)) {//check file exists or not
|             @unlink($image_path); //unlink image if image exist unlink(file path)
|             return redirect()->route('user.index'); //redirect to index route
|         }
|         return 'file not exists'; //error message if file not exists
|   }
```

7 App\Http\Controllers\UserController.php

```
|-------------------------------------------------------------------------
| UPDATE image on server & path database, update(form data, user id)
|-------------------------------------------------------------------------
   public function update(Request $request, string $id)
     {
         $user = User::find($id); //fetch data related to user $id

         if ($request->hasFile('photo')) {//check in request have any file or not

             $image_path = public_path('storage/images/') . $user->file_name;
             if (file_exists($image_path)) {//check file already exists or note
                 @unlink($image_path); //unlink old file if exists
             }

             $file = $request->file('photo'); //image data in photo key in file
             $fileName = time() . '_' . $file->hashName(); //generate has name
             $file->storeAs('images', $fileName, 'public'); //move file on server

             $user->whereId($id)->update([//update new image path on database
                 'file_name' => $fileName
             ]);
             return redirect()->route('user.index'); //redirect to index rote
         }
|-------------------------------------------------------------------------

|-------------------------------------------------------------------------
| Get File Original name
|-------------------------------------------------------------------------
       $file = $request->file('photo'); //image data in photo key in file
       $fileName = $file->getClientOriginalName();
       return $path;
|-------------------------------------------------------------------------
| Get File Extension name using extension() and getClientOriginalExtension();
|-------------------------------------------------------------------------
       $file = $request->file('photo');
       $extention = $file->extension();
       return $extention;
|-------------------------------------------------------------------------
| Generate has name for file path
|-------------------------------------------------------------------------
       $file = $request->file('photo');
       $path = $file->hashName();
       return $path;
|-------------------------------------------------------------------------
| Check file mime type (file type)
|-------------------------------------------------------------------------
       $file = $request->file('photo');
       $type = $file->getClientMimeType();
       return $type;
|-------------------------------------------------------------------------
| Check File size
|-------------------------------------------------------------------------
       $file = $request->file('photo');
       $file_size = $file->getSize();
       return $file_size;
|-------------------------------------------------------------------------
```

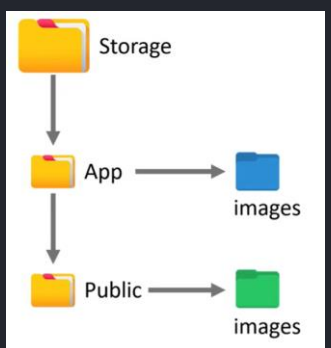
7 App\Http\Controllers\UserController.php

```
|--------------------------------------------------------------------------
| Get File Original name
|--------------------------------------------------------------------------
    public function store(Request $request)
    {
      $file = $request->file('photo'); //image data in photo key in file

      $request->validate([//validate request photo key
            'photo' =>
 'required|mimes:png,jpeg,jpg|max:5000|dimensions:min_width:10,min_height:10px'
      ]);

      $path = $request->file('photo')->store('images', 'public');
      //upload file using store method, method no need file name
      // file(key)->store(folder name , inside Public or Local folder)
      return $path;
    }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| UPLOAD file using storeAs method & file original name
|--------------------------------------------------------------------------
      $file = $request->file('photo');
      $request->validate(['photo' => 'required|mimes:png,jpeg,jpg|max:5000']);
      $fileName = time() . '_' . $file->getClientOriginalName();
      $path = $request->file('photo')->storeAs('images', $fileName, 'public');
      return $path;
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| UPLOAD file using move method (this method can store file any where)
|--------------------------------------------------------------------------
      $file = $request->file('photo'); //image data in photo key in file
      $fileName = time() . '_' . $file->hashName(); //generate has name for path
      $file->move(public_path('uploads'), $fileName);
      $file->move(public_path(folder name), file name);
      storage_path(storage folder path),resource_path(resourece folder path)
      User::create([
          'file_name' => $filename //add file name in to database
      ]);
      return redirect()->route('user.index'); //redirect to index route
|--------------------------------------------------------------------------
```

8 TABLE Migration code

```
Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('file_name');
            $table->timestamps();
        });
```

id	file_name
17	1723741551_mP731uxobYGPAfvbT8kLVatNotPBwb4l17xy4yi...
18	1723741559_k3kdoed132mDOSEz9gsJxbxLWuJrq99ZI43KMZp...



Model Accessors & Mutators

1 **php artisan make:model User --controller**

2 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User; //include model file
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function addUser() //custom method
    {
        $user = User::create([
            'user_name' => 'Ishika SAGAR',
            'email' => 'Ishika@EMAIL.com',
            'salary' => 5000,
            'dob' => '2024-08-21',
            'password' => '33249808324'
        ]);
        return redirect()->route('user.show');
    } //in this method insert User Data and redirect to home route

    public function showUser() //custom method
    {
        $users = User::get();
        return $users;
    } //in this method return all user data
}
```

3 Routes\web.php

```php
<?php

use App\Http\Controllers\UserController; //include Controller file
use Illuminate\Support\Facades\Route;

Route::get('/user/add', [UserController::class, 'addUser']); //route for add user
Route::get('/user/show', [UserController::class, 'showUser'])->name('user.show');
```

4 TABLE Migration code

```php
Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('user_name');
            $table->string('email')->unique();
            $table->integer('salary');
            $table->date('dob');
            $table->string('password');
        });
```

id	user_name	email	salary	dob	password
1	Dr. Godfrey Stanton DVM	kailey36@example.COM	1770	2008-11-13	OQzbjE5aVn
2	dion tillman	gmann@EXAmple.org	4677	2010-03-27	bGm4dvb2Oq
3	emmett williamson	juliana.haley@example.net	4092	2017-02-03	N2nWhOdiFa
4	Miss jade gerlach	antonio12@example.NET	1950	2021-10-30	Wyh4xP88no
5	Anjesh Kumar	anjesh@email.com	2000	2024-08-21	fsdfldsfdsof
6	abhiyansh kumar	abhiyansh@email.com	5000	2024-08-21	fsdfldsfdsof

Model Accessors & Mutators-II

5 App\Models\User.php

```php
<?php
namespace App\Models;
use Illuminate\Support\Number; //insert number class
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Casts\Attribute; //include attribute class
use Illuminate\Database\Eloquent\Factories\HasFactory;

class User extends Model
{
    use HasFactory;
    public $timestamps = false; //if your don't use timestamp column in your table
    protected $guarded = []; //if you allow all  column to insert or update data3

    public function setEmailAttribute($value) //method for email column
    {
        $this->attributes['email'] = strtolower($value);
    } automatic convert email string in to lower case while inserting new data


    public function setUserNameAttribute($value) //method for User Name column
    {
        $this->attributes['user_name'] = strtolower($value);
    } automatic convert name to lower case while inserting new data


    public function setPasswordAttribute($value) //method for password column
    {
        $this->attributes['password'] = bcrypt($value);
    } automatic convert password in to hash while inserting new data


    public function getDobAttribute($value) //method for DOB column
    {
        return date('D M Y', strtotime($value));
    } automatic convert date in readable format after fetching data from database


    public function getUserNameAttribute($value) //method for user name
    {
        return ucwords($value);
    } automatic convert name in to lower case after fetching data form database


    public function getSalaryAttribute($value) //method for salary column
    {
        return Number::currency($value, in: 'INR');
    } automatic convert number to currency format after fetching data


    protected function UserName(): Attribute //method for user name column
    {
        return Attribute::make(
            get: fn(string $value) => ucwords($value),
            set: fn(string $value) => strtolower($value)
        );
}    } automatic format the name columns data while fetching or inserting data
```

Authentication Register & Login (View Files)

1 **php artisan make:model User --controller**

2 resources\views\welcom.blade.php

```
<h1>Home Page</h1>
//2 link for register & Login Page
<a href="{{route('register')}}">Register</a>
<a href="{{route('login')}}">Login</a>
```

3 resources\views\inner.blade.php

```
<h1>Inner Page</h1>
//check Login user name
@if (auth()->check())
    {{Auth::user()->name}}
@endif
```

4 resources\views\register.blade.php

```
<h1>Register</h1>

<form action="{{route('register.save')}}" method="POST">
    @csrf
    <label for="username">Name</label>
    <input type="text" id="username" name="name">
    <label for="useremal">Email</label>
    <input type="text" id="useremail" name="email">
    <label for="userpassword">Password</label>
    <input type="password" id="userpassword" name="password">
    <label for="password-confirm">Confirm Password</label>
    <input type="password" id="password-confirm" name="password_confirmation">
    <button type="submit">Register</button>
</form> //submit form data to register.Save rote, password confirmation name should
be password_confirmation, and user csrf token

@if ($errors->any()) //check any error in this page add this code to all view file
<ul>
    @foreach ($errors->all() as $error) <li>{{$error}}</li> @endforeach
</ul>
@endif
```

5 resources\views\login.blade.php

```
<h1>Login</h1>

<form action="{{route('loginMatch')}}" method="POST">
    @csrf
    <label for="useremal">Email</label>
    <input type="text" id="useremail" name="email">
    <label for="userpassword">Password</label>
    <input type="password" id="userpassword" name="password">
    <button type="submit">Login</button>
</form> //submit login detail to Login Match route
```

6 resources\views\login.blade.php

```
<h1>Dashboard Page : Welcome {{Auth::user()->name}}</h1> //show login user name
{{Auth::user()}} //you can show any detail of Authenticated user, id, name, email

<a href="{{route('innerPage')}}">Inner Page</a> //link for inner page
<a href="{{route('logout')}}">Logout</a> //link for logout route for logout user
```

9 App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\User; //include model file
use Auth; //include auth class
use Illuminate\Http\Request; //include request class

class UserController extends Controller
{
    public function homePage() //method for home page
    {
        if (Auth::guest()) {//check visitor is guest or login user
            return view('welcome');
        } else {
            return redirect()->route('dashboard');
        }
    } //in this method if your not login return welcome (home page) else dashboard

    public function register(Request $request)
    {
        $data = $request->validate([//validate the form filed
            'name' => 'required',
            'email' => 'required|email',
            'password' => 'required|confirmed'
        ]);
        $user = User::create($data); //insert new data to user table

        if ($user) {
            return redirect()->route('login'); //redirector login route
        }
    } //method for register new user and redirect to login route

    public function login(Request $request) //method for authenticate the user
    {
        $creadentials = $request->validate([
            'email' => 'required|email',
            'password' => 'required'
        ]);
        if (Auth::attempt($creadentials)) {//login the user using Auth::attempt
            return redirect()->route('dashboard');
        }
    } //in this method validate user data and authorize the user (create session)

    public function dashboardPage() //show dashboard page
    {
        if (Auth::check()) {//using check method to check user login or not
            return view('dashboard');
        } else {
            return redirect()->route('login');
        }
    } //check if user login then return dashboard page else redirect to login page

    public function logout() //logout the user
    {
        Auth::logout();
        return redirect()->route('login');
    } //using Auth::logout() to logout the user and redirect to login page
}
```

7 App\Models\User.php

```php
<?php

namespace App\Models;
use Illuminate\Foundation\Auth\User as Authenticatable; //include Auth Class
•  use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Authenticatable //extend Authenticable class
{
    use HasFactory;
    public $timestamps = false; //if in your table don't have crated_at column
    protected $fillable = ['name', 'email', 'password']; //allow to fill data column
    protected $hidden = ['id', 'password']; //hide column when fetch data

    protected function casts(): array
    {
        return ['password' => 'hashed']; //when new data insert this method
    }                    automatically convert password in hash before data create
}
```

8 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //for login page
use Illuminate\Support\Facades\Route;
//call homepage Method
Route::get('/home', [UserController::class, 'homePage']);
//return register page
Route::view('register', 'register')->name('register');
//call register method on registerSave route
Route::post('/registerSave', [UserController::class, 'register'])->name('register.save');
Route::view('login', 'login')->name('login'); //return login page for login route
//call loginMath method on loginMatch route
Route::post('/loginMatch', [UserController::class, 'login'])->name('loginMatch');
//call dashboardPage method on dashboard route
Route::get('/dashboard', [UserController::class, 'dashboardPage'])->name('dashboard');
//call innerPage method for inner route to show inner page
Route::get('/inner', [UserController::class, 'innerPage'])->name('innerPage');
//call logout method for logout route
Route::get('/logout', [UserController::class, 'logout'])->name('logout');
```

App\config\Auth.php

```php
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
], //if your table name different
from user, replace users to table
```

Migration Code

```php
Schema::create('users', function (Blueprint $table)
{
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->string('password');
        }); //migration table code
```

The Laravel Auth class is a core component of Laravel's authentication system. Here's a bit more detail:

- **Login/Logout**: Handles user authentication with methods like attempt(), login(), and logout().
- **Check Authentication**: Methods like check() and guest() allow checking if a user is authenticated or not.
- **Current User**: user() retrieves the currently authenticated user, while id() gets the user's ID.
- **Middleware**: Auth integrates with middleware like auth and guest to protect routes.
- **Remember Me**: Supports "remember me" functionality with viaRemember().

Middleware With Authentication (View Files)

1 **php artisan make:model User --controller**

2 **php artisan make:middelware ValidUser**

3 resources\views\welcom.blade.php

```
<h1>Home Page</h1>
//2 link for register & Login Page
<a href="{{route('register')}}">Register</a>
<a href="{{route('login')}}">Login</a>
```

4 resources\views\inner.blade.php

```
<h1>Inner Page</h1>
@if (auth()->check())
    {{Auth::user()->name}}
@endif //check Login user name
```

5 resources\views\register.blade.php

```
<h1>Register</h1>

<form action="{{route('register.save')}}" method="POST">
    @csrf
    <label for="username">Name</label>
    <input type="text" id="username" name="name">
    <label for="useremal">Email</label>
    <input type="text" id="useremail" name="email">
    <label for="userage">Age</label>
    <input type="number" id="userage" name="userage">
    <label for="userrole">Role</label>
    <input type="text" id="userrole" name="role">
    <label for="userpassword">Password</label>
    <input type="password" id="userpassword" name="password">
    <label for="password-confirm">Confirm Password</label>
    <input type="password" id="password-confirm" name="password_confirmation">
    <button type="submit">Register</button>
</form> //submit form data to register.Save rote, password confirmation name should be
          password_confirmation, and user csrf token

@if ($errors->any()) //check any error in this page add this code to all view file
<ul> @foreach ($errors->all() as $error) <li>{{$error}}</li> @endforeach </ul>
@endif
```

6 resources\views\login.blade.php

```
<h1>Login</h1>
<form action="{{route('loginMatch')}}" method="POST">
    @csrf
    <label for="useremal">Email</label>
    <input type="text" id="useremail" name="email">
    <label for="userpassword">Password</label>
    <input type="password" id="userpassword" name="password">
    <button type="submit">Login</button>
</form> //submit login detail to Login Match route
```

7 resources\views\dashboard.blade.php

```
@if(Auth::check()) //check user login if user login the show user name
<h1>Dashboard Page : Welcome {{Auth::user()->name}}</h1>  //show user name using Auth::user()
    {{Auth::user()}} //show user all detail using Auth::user() method
@endif

<a href="{{route('innerPage')}}">Inner Page</a> //redirect to innerPage
<a href="{{route('logout')}}">Logout</a> //redirect to logout route
```

Middleware With Authentication (middleware Files)-II

8 App\Http\Middleware\ValidUser.php

```php
<?php
namespace App\Http\Middleware;
use Closure; //include closure calss
use Illuminate\Http\Request; //include request class
use Illuminate\Support\Facades\Auth; //include auth class
use Symfony\Component\HttpFoundation\Response; //include response class

class ValidUser
{  //in handle method we check user is validate using Auth::check() method if your valid
    then return request using next method, else redirect to login page
    public function handle(Request $request, Closure $next): Response
    {
        echo '<h1>now you are in Valid User Page</h1>';
        if (Auth::check()) {
            return $next($request);
        } else {
            return redirect()->route('loginPage');
        }
    }

//2nd way to define middleware in handle method 3rd parameter is dynamic value get from route

    public function handle(Request $request, Closure $next, string $role): Response
    {
        echo '<h1>now you are in Valid User Page</h1>';
        if (Auth::check() && Auth::user()->role == $role) {//check user is admin & valid
            return $next($request); //return request if user pass both condition
        } else {
            return redirect()->route('loginPage'); //if user not pass both condition then
        }                                              redirect to login page
    }
    public function terminate(Request $request, Response $response): void {
        echo '<h1>Terminating the User</h1>';
    } //this method run after $next() method end of method
}
```

9 Bootstrap\app.php

```php
->withMiddleware(function (Middleware $middleware) {

        $middleware->alias([
            'testMiddleware' => Testuser::class,
            'ValidUser' => ValidUser::class
        ]); //re-name the middleware using alias() method


        $middleware->appendToGroup('ok-user', [
            ValidUser::class,
            Testuser::class
        ]); //make multiple middleware group appendToGroup(group name) name use in route


        $middleware->append(TestUser::class); //fist way to define global middleware
        $middleware->use([
            TestUser::class,
            ValidUser::class
        ]); //second way to define global middleware, global middleware run for all route
    })
```

middleware handles user authentication and role checking, redirecting unauthenticated users to the login page It includes a terminate method that runs after the response is sent. Middleware

Middleware With Authentication (route & model)-III

10 Routes\web.php

```php
<?php
use App\Http\Controllers\UserController; //include all middleware & controller class
use App\Http\Middleware\Testuser;
use App\Http\Middleware\ValidUser;
use Illuminate\Support\Facades\Route;

Route::controller(UserController::class)->group(function () {//group route for UserController
    Route::get('/', 'homePage')->name('homePage'); //call homePage method
    Route::get('/login', 'loginPage')->name('loginPage'); //call loginPage method
    Route::get('/register', 'registerPage')->name('registerPage'); //callresitePage method
    Route::post('/registersave', 'register')->name('register.save'); //call register method
    Route::post('/loginmatch', 'login')->name('loginMatch'); //call login method
    Route::get('/logout', 'logout')->name('logout'); //call logout method
});


Route::middleware([ValidUser::class, Testuser::class])->group(function () {
    Route::get('/dashboard', 'dashboardPage')->name('dashboard');
    Route::get('/dashboard/inner', 'innerPage')->name('innerPage')-
>withoutMiddleware(Testuser::class); //if you don't want user Testuser middleware
}); //make middleware group to call multiple middlware


Route::withoutMiddleware(Testuser::class)->group(function () {
    Route::get('/dashboard/inner', 'innerPage')->name('innerPage');
}); //if you use multiple middleware but any middleware you don't want use for this route

Route::middleware(['ok-user'])->group(function () {
    Route::get('/dashboard', [UserController::class, 'dashboardPage'])->name('dashboard');
    Route::get('/dashboard/inner', [UserController::class, 'innerPage'])->name('innerPage');
}); //we make multiple middleware group in Validuser.php user group name as middleware name


Route::middleware(['ValidUser:admin'])->group(function () {
    Route::get('/dashboard', [UserController::class, 'dashboardPage'])->name('dashboard');
    Route::get('/dashboard/inner', [UserController::class, 'innerPage'])->name('innerPage');
}); //we user re-name version of middleware and pass value to middleware after ":" :admin
```

11 App\Models\User.php

```php
<?php
namespace App\Models;
use Illuminate\Foundation\Auth\User as Authenticatable; //include User as Authenticatable
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Authenticatable //extend Authenticatable class
{
    use HasFactory;

    public $timestamps = false; //if your table don't have create_at & update_at column

    protected $fillable = ['name', 'email', 'password', 'userage', 'role']; //allow fill data

    protected $hidden = ['id', 'password',]; //hide column when data fetch

    protected function casts(): array {
        return ['password' => 'hashed','userage' => 'integer'];
    } //this method run automatic convert password column into hash,
}       example: column name => method,
```

12 App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\User; //include User Model
use Auth; //include auth class
use Illuminate\Http\Request; //include Request class

class UserController extends Controller
{
    public function homePage(){ return view('welcome'); } //return welcome view
    public function loginPage()  { return view('login'); } //return login view
    public function dashboardPage() { return view('dashboard'); } //return dashboard view
    public function innerPage() { return view('inner'); }  //return inner view
    public function registerPage() { return view('register'); } //return register view

-----------------------------------------------------------------------------------------

    public function register(Request $request) //custom method for register user
    {
        $data = $request->validate([//validate the user
            'name' => 'required',
            'email' => 'required|email',
            'userage' => 'required|integer',
            'role' => 'required',
            'password' => 'required|confirmed'
        ]);
        $user = User::create($data); //insert user data to user table

        if ($user) {

            return redirect()->route('loginPage');
        } //check and redirect when user data successfully insert
    }


-----------------------------------------------------------------------------------------


    public function login(Request $request) //custom method for authenticate the user
    {
        $credentials = $request->validate([//validate user details
            'email' => 'required|email',
            'password' => 'required'
        ]);
        if (Auth::attempt($credentials)) {//authenticate user using Auth::attempt
            if (Auth::check()) {
                return redirect()->route('dashboard'); //return dashboard if user login
            } else {
                return redirect()->route('loginPage'); //if user not authenticate then return
               }                                                                    login page
            } else {
            return redirect()->route('loginPage'); //if user not authenticate then return
        }                                                                       login page
    }

-----------------------------------------------------------------------------------------


    public function logout() //custom method to logout the user
    {
        Auth::logout(); //logout the user using Auth::logout() method
        return redirect()->route('loginPage'); //when user logout return to login page
    }
}
```

Session Store, Show, Delete

1 **php artisan make:controller UserController**

2 App\Http\Controllers\UserController.php

```php
public function show_session(){
|--------------------------------------------------------------------------
| here is 3 way to show session value session(Key) (return array)
|--------------------------------------------------------------------------
|     $value = session()->get('name'); //read session using get(key name) method
|     echo $value;
|
|     $value = session('lang'); //read session using session method(key name)
|     echo $value;
|
|     $value = session('name', 'hello'); //read multiple session(key1,key2) method
|     echo $value;
|
|     $value = session()->all();
|     echo '<pre>'; print_r($value); echo '</pre>'; //read all session
|--------------------------------------------------------------------------
|  Except method show all session except(key) key name pass in method
|--------------------------------------------------------------------------
|     $value = session()->except(['name']);
|     echo '<pre>'; print_r($value); echo '</pre>';
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Only method show only([key name]) key name pass in method
|--------------------------------------------------------------------------
|     $value = session()->only(['name']);
|     echo '<pre>'; print_r($value); echo '</pre>';
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Has method use for check session key & value exist or not has(key name)
|--------------------------------------------------------------------------
|     if (session()->has('name')) {
|         $value = session()->get('name');
|         echo $value;
|     } else {
|         echo 'name key dosnot exists';
|     }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| Exist method user for check session key exists or not exists(Key name)
|--------------------------------------------------------------------------
|     if (session()->exists('name')) {
|         $value = session()->get('name');
|         echo $value;
|     } else {
|         echo 'name key dosnot exists';
|     }
|
|--------------------------------------------------------------------------
 }
```

Session Store, Show, Delete-II

2 App\Http\Controllers\UserController.php

```php
|--------------------------------------------------------------------------
| STORE session here is 2 way to store session
|--------------------------------------------------------------------------
|     public function store_session() {
|
|     session(['name' => 'Yahoo Baba']); //store ingle session
|
|     session([
|         'name' => 'Yahoo Baba', //store multiple session
|         'lang' => 'laravel'
|     ]);
|
|     session()->put('name', 'Yahoo Baba'); //store single session using put method
| }
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| STORE session helper method help dewing storing session
|--------------------------------------------------------------------------
|    //using flash method to store temporarily session and send to view
|     session()->flash('status', 'session saved successfully.');
|     return redirect('/session/show');
|
|     session()->increment('count', $incrimentBy = 2); //increase key value
|     session()->decrement('count', $decrimentby = 2); //decrees key value
|     session()->regenerate(); //regenerate session token use in read & create time
|--------------------------------------------------------------------------

|--------------------------------------------------------------------------
| DELETE session method here 3 way to delete session
|--------------------------------------------------------------------------
|     public function delete_session()
|     {
|         session()->forget('name'); //single session forget(key name)
|         session()->forget(['name', 'lang']); //multiple session
|         session()->flush(); //delete all session
|         session()->invalidate(); //delete session token useful method
|     }
|--------------------------------------------------------------------------
```

3 Routes\web.php

```php
<?php

use App\Http\Controllers\SessionController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});
Route::get('/session/show', [SessionController::class, 'show_session']);
Route::get('/session/store', [SessionController::class, 'store_session']);
Route::get('/session/delete', [SessionController::class, 'delete_session']);
```

Gate (View Files)

1 **php artisan make:model User --controller**

2 **php artisan make:model Post**

3 resources\views\login.blade.php

```
<h1>Login</h1>
<form action="{{route('loginMatch')}}" method="POST">
    @csrf
    <label for="useremal">Email</label>
    <input type="text" id="useremail" name="email">
    <label for="userpassword">Password</label>
    <input type="password" id="userpassword" name="password">
    <button type="submit">Login</button>
</form> //submit login detail to Login Match route

@if ($errors->any()) //check any error in this page add this code to all view file
<ul> @foreach ($errors->all() as $error) <li>{{$error}}</li> @endforeach </ul>
@endif
```

4 resources\views\post.blade.php

```
<h1>Post Page</h1>
<table> //show post related to login user
<tr>
<th>id</th><th>title</th><th>description</th><th>user_id</th><th>Update</th><th>Delete</th>
 </tr>
 @foreach ($posts as $post)
    <tr>
         <td>{{$post->id}}</td>
         <td>{{$post->title}}</td>
         <td>{{$post->description}}</td>
         <td>{{$post->user_id}}</td>
         <td><a href="{{route('post.update',$post->id)}}">Update</a></td>
        //redirect to update page with post id
    </tr>
 @endforeach
 </table>
```

5 resources\views\dashboard.blade.php

```
@if (Gate::allows('isAdmin'))
<a href="" style="color: green">Admin Pannel</a> //show this link only if user admin
@endif

@can('isAdmin')
<a href="" style="color: green">Admin Pannel</a> //2nd way show this link only if user admin
@else
<h1>You Are Not Admin</h1> //if user not admin show this message
@endcan

@cannot('isAdamin')
<a href="" style="color: green">Other Pannel</a>    //if user not admin show this link
@endcannot

<a href="{{route('user.profile',Auth::id())}}" style="color: rgb(66, 27, 139)">Profile</a>
<a href="{{route('post.show',Auth::id())}}" style="color: chocolate">Post</a>
<a href="{{route('logout')}}" style="color: red">Logout</a>
```

Gate (Gate & route)-II

6 App\Providers\AppServiceProviders.php

```php
<?php
namespace App\Providers;
use App\Models\User; //include User Class
use Illuminate\Support\Facades\Gate; //include Gate class
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{

    public function boot(): void //define your all gate in boot method
    {
        Gate::define('isAdmin', function (User $user) {
            return $user->role === 'admin'; //gate to check user is admin or not
        });

        Gate::define('view-profile', function (User $user, $UserId) {
            return $user->id === $UserId; //gate for check login user & given user id match
        });

        Gate::define('update-post', function (User $user, $targetUser) {
            return $user->id === $targetUser; //gate for check user update there on post
        });

        Gate::before(function (User $user) {
                //this method run before all gate,
        });
    }
}       //in gate method also have after() method run after all gate
```

7 web.php

```php
<?php
use App\Http\Controllers\UserController;
use App\Http\Middleware\ValidateUser;
use Illuminate\Support\Facades\Route;

Route::get('/login', [UserController::class, 'loginPage'])
->name('loginPage'); //route for login page

Route::post('/loginMatch', [UserController::class, 'loginMatch'])
->name('loginMatch'); //route for authenticate login method

Route::get('/dashboard', [UserController::class, 'dashboardPage'])->name('dashboard')
->middleware('can:isAdmin'); //route for dashboard page and user user isAdmin gate

Route::get('/dashboard', [UserController::class, 'dashboardPage'])
->name('dashboard')->can('isAdmin'); //2nd way to user gate in route using can() method

Route::get('/profile/{id}', [UserController::class, 'profilePage'])->name('user.profile')
->can('isAdmin'); //show user profile, and user isAdmin gate to restrict none admin user

Route::get('/post/{id}', [UserController::class, 'showPost'])
->name('post.show'); //route for show all post page

Route::get('/single-post/{id}', [UserController::class, 'postUpdate'])
->name('post.update'); //route for show single user post

Route::get('/logout', [UserController::class, 'logout'])
->name('logout'); //route for logout page
```

Gate (Controller)-III

8 App\Http\Controllers\UserController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Post; //include post model file
use App\Models\User; //include user model file
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Gate;

class UserController extends Controller
{
    public function loginPage()
    {
        return view('login'); //return login page
    }

-----------------------------------------------------------------------------------------------
    public function loginMatch(Request $request)
    {
        $credential = $request->validate([//validate email and password
            'email' => 'required|email',
            'password' => 'required',
        ]);

        if (Auth::attempt($credential)) {//Authenticate the user using Auth::attempt() method
            if (Auth::check()) {
                return redirect()->route('dashboard'); //check if user login the redirect to
            } else {                                        dashboard page
                return redirect()->route('loginPage');
            }
        } else {
            return redirect()->route('loginPage'); //if your not Authenticate then redirect
        }                                             to login page
    }

-----------------------------------------------------------------------------------------------
    public function dashboardPage()
    {
        Gate::authorize('isAdmin'); //check user is admin or not using Gate::authorize method
        return view('dashboard');

        if (Gate::allows('isAdmin')) {//2nd way to check user admin or not using allows()
            return view('dashboard'); //if user admin then return dashboard page
        } else {
            return 'Access deinei'; //if user not admin then return message or error page
        }

    } //isAdmin is Gate name we define in gate file

-----------------------------------------------------------------------------------------------
    public function profilePage(int $userID) //method for profile page show user info
    {
        if (Gate::allows('view-profile', $userID)) {//check login user and given user id same
            $user = User::findOrFail($userID);//if gate condition match then fetch user data
            return $user;
        } else {
            return 'Wrong user id'; //if condition not match then return massage or view
        }

        Gate::authorize('view-profile', $userID); // 2nd way to user gate method
        $user = User::findOrFail($userID);
        return $user;
    } //in authorize(),allows() method second parameter pass user id to match Id in Gate file
```

Gate (Controller)-IV

8 App\Http\Controllers\UserController.php

```
    public function showPost() //method to show post relate to login user
    {
        $posts = Post::where('user_id', Auth::id())->get(); //where login user id & user_id
        return view('post', compact('posts'));                                  in post table
    }
-------------------------------------------------------------------------------------------
    public function postUpdate($postId) //method for update user page
    {
        $post = Post::find($postId); //find user by given post id

        $userId = $post->user_id; //extract post user id column name user_id

        Gate::authorize('update-post', $userId); //then use gate to check login user or
                                                                    given user id match
        $post = Post::findOrFail($postId);
        return $post; //if gate return true then show fetch post data related to user
    }
-------------------------------------------------------------------------------------------
    public function logout() //logout method to logout the user
    {
        Auth::logout();
        return redirect()->route('loginPage'); //if user logout the redirect to login page
    }
}
Gate::any(['view-profile']) any method return true if both or one condition match
Gate::none(['view-profile']) none only return true if all Gate condtion not match
```

9 App\Models\User.php

```
<?php
namespace App\Models;
use Illuminate\Foundation\Auth\User as Authenticatable; //include User as Authenticatable
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Authenticatable //extend Authenticatable class
{
    use HasFactory;

    public $timestamps = false; //if your table don't have create_at & update_at column

    protected $fillable = ['name', 'email', 'password', 'userage', 'role']; //allow fill data

    protected $hidden = ['id', 'password',]; //hide column when data fetch

    protected function casts(): array {
        return ['password' => 'hashed','userage' => 'integer'];
    } //this method run automatic convert password column into hash,
}       example: column name => method,
```

10 App\Http\Middleware\ValidUser.php

```
    public function handle(Request $request, Closure $next): Response
    {
        if (Auth::check()) {//define middle ware to check user is login or not
            return $next($request);
        } else {
            return redirect()->route('loginPage'); //if user not login then return loginpage
        }
```

Policy (View Files)

1. **php artisan make:model Book –controller --resource**

2. **php artisan make:model User –controller**

3. **php artisan make:policy BookPolicy --model=Book (Policy name fist model name then prefix Policy)**

4. resources\views\login.blade.php

```
<h1>Login</h1>
<form action="{{route('login')}}" method="POST">
    @csrf
    <label for="useremal">Email</label>
    <input type="text" id="useremail" name="email">
    <label for="userpassword">Password</label>
    <input type="password" id="userpassword" name="password">
    <button type="submit">Login</button>
</form> //submit login detail to Login Match route

@if ($errors->any()) //check any error in this page add this code to all view file
<ul> @foreach ($errors->all() as $error) <li>{{$error}}</li> @endforeach </ul>
@endif
```

5. resources\views\book_index.blade.php

```
<h1>Books Page</h1>
<h3> @if (Auth::check()) {{Auth::user()->name}} @endif</h3> //if user login show user name

    <a href="{{route('books.create')}}">Create Book</a> //link redirect to book.create route
    <a href="{{route('logout')}}">Logout</a> //link redirect to logout route

    <table><tr><th>id</th><th>Title</th><th>Price</th><th>User ID</th></tr>
        @foreach ($Books as $Book) //display all book table data using foreach loop
        <tr>
            <td>{{$Book->id}}</td>
            <td>{{$Book->title}}</td>
            <td>{{$Book->price}}</td>
            <td>{{$Book->user_id}}</td>
            <td>

            @if (Auth::user()->can('update',$Book)) //fist way to user policy in view
                  //using can method to call policy and pass book data to check book user_id
            @endif //same login id then show update button


            @can('update', $Book) //2nd way to call policy method to determinate user
             {{-- @canany(['update','view'], $Book) --}} //user canany for multiple Policy
            <a href="{{route('books.show',$Book->id)}}">View</a>
            <a href="{{route('books.edit',$Book->id)}}">Update</a>
            <a href="">Delete</a></td>
            @else
            <h6>You are not Authrized.</h6> //if book user_id and login user id match then
            @endcan //then show update,edit,delte link, using can method to call policy

            @unless (Auth::user()->can('update',$Book)) //if user don't have access then
                //if login user and book user_id match and login user don't have permission
            @endunless    //then show massage or containt
        </tr>
        @endforeach
    </table>
```

6 resources\views\book_create.blade.php

```
<h1>Creat New Book Recode</h1> // create new book recode

<form action="{{route('books.store')}}" method="POST"> //submit data to book.store route
    @csrf

    <label for="title">Book Title</label>
    <input type="text" id="title" name="title">
    <label for="price">Book Price</label>
    <input type="text" id="price" name="price">
    <button type="submit">Save</button>
</form>
```

7 resources\views\book_show.blade.php

```
<h1>Book Show</h1> //show book detail

<h3>{{$book->title}}</h3>
<h3>{{$book->price}}</h3>
```

8 resources\views\book_update.blade.php

```
<h1>Update Book Reocde</h1> //update book recode

<form action="{{route('books.update',$book->id)}}" method="POST"> //data submit to
    @csrf                                                          book.update route
    @method('PUT')
    <label for="title">Book Title</label>
    <input type="text" id="title" name="title" value="{{$book->title}}">
    <label for="price">Book Price</label>
    <input type="text" id="price" name="price" value="{{$book->price}}">
    <button type="submit">Save</button>
</form>
```

9 web.php

```
<?php

use App\Http\Controllers\BookController; //include Book Controller file
use App\Http\Controllers\UserController; //include User Controller file
use GuzzleHttp\Middleware; //include Middleware class
use Illuminate\Support\Facades\Route;

Route::get('/', [UserController::class, 'loginPage'])->name('home');
//route for show login page call login page method of User Controller

Route::post('/login', [UserController::class, 'login'])->name('login');
//route for login call login method of User Controller

Route::post('/logout', [UserController::class, 'logout'])->name('logout');
//route for logout call logout method of User Controller

Route::resource('books', BookController::class)->middleware('auth');
//resource route for Book Controller and add middleware for Authenticate user is login or not

Route::put('/book/update/{id}', [BookController::class, 'update']) //use policy in this route
    ->can('update','view'); //using can method to add single and multiple policy
    ->Middleware('can:update', 'view'); //2nd way to call policy using middleware(can:policy)
```

Policy (User Controller & User Model)-III

10 App\Http\Controllers\UserController.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request; //include Request Class
use Illuminate\Support\Facades\Auth; //include Auth Class

class UserController extends Controller
{
    public function loginPage() //method return login page
    {
        if (Auth::check()) {//check if user login then redirect to books.index route
            return redirect()->route('books.index');
        }
        return view('login'); //if user not login then return login view(login page)
    }
-----------------------------------------------------------------------------------------

    public function login(Request $Request) //method for Authenticate the user (login)
    {
        $credential = $Request->validate([//validate the form request data
            'email' => 'required|email',
            'password' => 'required'
        ]);
        if (Auth::attempt($credential)) {//using Auth::attempt() method to login the user
            return redirect()->route('books.index'); //if user login then redirect to
        }                                                                  book.index route
        return redirect()->back(); //if user not login then redirect to back page
    }
-----------------------------------------------------------------------------------------

    public function logout() //method for logout the user
    {
        Auth::logout(); //logout the user using Auth::logout() method
        return redirect()->route('home'); //if user logout then return home page
    }                                     //tips after logout the user regenerate the session
}                                                                                  token
```

11 App\Models\User.php

```php
<?php
namespace App\Models;
use Illuminate\Foundation\Auth\User as Authenticatable; //include User as Authenticatable
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class User extends Authenticatable //extend Authenticatable class
{
    use HasFactory;

    public $timestamps = false; //if your table don't have create_at & update_at column

    protected $fillable = ['name', 'email', 'password', 'userage', 'role']; //allow fill data

    protected $hidden = ['id', 'password',]; //hide column when data fetch

    protected function casts(): array {
        return ['password' => 'hashed','userage' => 'integer'];
    } //this method run automatic convert password column into hash,
}       example: column name => method,
```

12 App\Policies\BookPolicy.php

```php
<?php
namespace App\Policies;
use App\Models\Book; use App\Models\User; //include User & Book model file
use Illuminate\Auth\Access\Response;

class BookPolicy //make policy method to determinate the user in : view, controller, route
{
    public function view(User $user, Book $book): bool{
        return $user->id === $book->user_id; //check login user id & book user_id is same ?
    }

    public function edit(User $user, Book $book): bool{
        return $user->id === $book->user_id; //check login user id & book user_id is same ?
    }

    public function update(User $user, Book $book): bool{
        return $user->id === $book->user_id; //check login user id & book user_id is same ?
    }

    public function delete(User $user, Book $book): bool{
        return $user->id === $book->user_id; //check login user id & book user_id is same ?
    }

    public function before(User $user): bool|null
    {
    // this method run before all method make any condition like chec user is login or note
    }
    public function forceDelete(User $user, Book $book): bool{
    //make condition when data force fully delete
    }
    public function create(User $user): bool{
    //make condition when user create new data
    }
    public function restore(User $user, Book $book): bool{
    } //make condition when user restore the data
} *NOTE* : all Policy method return bool value true or false
//if your policy defret like you don't user Policy prefex then
//register your model and policy file in AppServiceProvider.php
```

App\Providers\AppServiceProvider.php

```php
use App\Models\Order;
use App\other-folder\OrderPolicy;
use Illuminate\Support\Facades\Gate;

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Gate::policy(Order::class, OrderPolicy::class);
    }
}
```

13 TABLE Migration code

```php
Schema::create('books', function (Blueprint $table) {//book table migration code
            $table->id();
            $table->string('title');
            $table->integer('price');
            $table->unsignedBigInteger('user_id');
            $table->foreign('user_id')->references('id')->on('users');
            $table->timestamps();
        });

Schema::create('users', function (Blueprint $table) {//user table migration code
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->integer('userage');
            $table->string('role');
            $table->string('password');
        });
```

Policy (Book Controller)-V

14 App\Http\Controllers\BookController.php

```php
<?php
namespace App\Http\Controllers;
use App\Models\Book; //include Book Model file
use Auth; use Gate; //include Auth & Gate Class
use Illuminate\Http\Request;

class BookController extends Controller
{
    public function index() //in this method fetch all book data & pass to index view file
    {
        $Books = Book::all(); //fetch all data from book table
        return view('books_index', compact('Books')); //return book_index view with book data
    }
-----------------------------------------------------------------------------------------------

    public function create() //in this method return book.create view file
    {
        return view('books_create');
    }
-----------------------------------------------------------------------------------------------

    public function store(Request $request)
    {
        $request->validate(['title' => 'required','price' => 'required|numeric']);
        $user_id = Auth::id();

        Book::create([
            'title' => $request->title,
            'price' => $request->price,
            'user_id' => $user_id
        ]);
        return redirect()->route('books.index');
    }
-----------------------------------------------------------------------------------------------

    public function show(Book $book)
    {
        Gate::authorize('view', $book);
        return view('book_show', compact('book'));
    }
-----------------------------------------------------------------------------------------------
    public function edit(Book $book)
    {
        Gate::authorize('edit', $book);
        $book = Book::findOrFail($book->id);
        return view('book_update', compact('book'));
    }
-----------------------------------------------------------------------------------------------

    public function update(Request $request, Book $book)
    {
        // Gate::authorize('update', $book);
        if ($request->user()->cannot('update', $book)) {
            abort(403, 'You are not Authrized');
        }
        $request->validate(['title' => 'required','price' => 'required|numeric']);

        Book::where('id', $book->id)->update([
            'title' => $request->title,
            'price' => $request->price
        ]);
}   }
```

Send Email (.env, route, view, contoller)

1 **php artisan make:controller EmailController**

2 **php artisan make:mail Welcomemail**

3 .env

```
MAIL_MAILER=smtp                          //smtp for gmail
MAIL_HOST=smtp.gmail.com                  //smtp.gmail.com for gmail
MAIL_PORT=587                             //587 port number for gmail
MAIL_USERNAME=anjeshravi8@gmail.com       //your username or mail id
MAIL_PASSWORD=fisvyknpxdjgajjq            //gmail app password, generate on google security
MAIL_ENCRYPTION=tls                       //tls encryption type for gmail
MAIL_FROM_ADDRESS="anjeshravi8@gmail.com" //send from mail id
MAIL_FROM_NAME="${APP_NAME}"
```

4 web.php

```
<?php
use App\Http\Controllers\EmailController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/send-email', [EmailController::class, 'SendEmail']); //call SendEmail method
```

5 resources\views\mail\welcome-email.blade.php

```
    <h3>{{$subject}}</h3>                   //show subject in mail
    <p>{{$mailmessage}}</p>                 //show message in mail
    <p>{{$details['name']}}</p>             //show name key of details array
    <p>{{$details['product']}}</p>          //show product key of details array
    <p>{{$details['price']}}</p>            //show price key of details array
-------------------------------------------------------------------------------------------
    <p>{{$name}}</p>       //show name if your data protected, send throw with method
    <p>{{$product}}</p>    //show product if your data protected, send throw with method
    <p>{{$price}}</p>      //show price if your data protected, send throw with method
```

6 App\Http\Controllers\EmailController.php

```
<?php
namespace App\Http\Controllers;
use App\Mail\Welcomeemail; //include welcome email class
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Mail; //include mail class

class EmailController extends Controller
{
    public function SendEmail() //method for send email
    {
        $toEmail = 'anjeshravi8@yahoo.com'; //mail id to send mail
        $message = 'Sending mail from Laravel Project'; //message send to in mail
        $subject = 'Test Email Send Throw Laravel'; //subject of mail
        $details = ['name' => 'Yahoo Baba', 'product' => 'Laravel', 'price' => 0];

        //send mail using mail class
        $request = Mail::to($toEmail)->send(new Welcomeemail($message, $subject, $details));
        // Mail::to($toEmail)->cc($mailid)->send(); //send to cc using cc() method
        // Mail::to($toEmail)->bcc($mailid); //send to bcc using bcc() method
        dd($request);
    }
}
```

Send Email (custom mailable class)-II

7 App\Mail\Welcomemail.php

```php
<?php

namespace App\Mail;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Mail\Mailable;
use Illuminate\Mail\Mailables\Content;
use Illuminate\Mail\Mailables\Envelope;
use Illuminate\Queue\SerializesModels;

class Welcomeemail extends Mailable
{
    use Queueable, SerializesModels;
-----------------------------------------------------------------------------------------------

    public $mailmessage; //initialize variable for massage send in mail
    public $subject;     //initialize variable for subject of mail
    public $details;     //initialize variable for details extra data send in mail
    protected $data;     //initialize protected variable if needed

-----------------------------------------------------------------------------------------------

    public function __construct($message, $subject, $details)
    {
        $this->mailmessage = $message; //assign $message to mailmessage variable
        $this->subject = $subject;     //assign $subject to subject variable
        $this->details = $details;     //assign $details to details variable
    }

-----------------------------------------------------------------------------------------------

    public function envelope(): Envelope //envelope method return subjec
    {
        return new Envelope(
            subject: $this->subject, //return subject variable
        );
    }

-----------------------------------------------------------------------------------------------


public function content(): Content //content method return view or text
    {
        return new Content(
            view: 'mail.welcom-mail', //return view file (mail/welcome-mail.blade.php)
            // text: 'mail.welcom-mail', //if only send text simple show variable in view
            with: [
                'name' => $this->data['name'],
                'product' => $this->data['product'],
                'price' => $this->data['price'],
            ] //if any variable protected then send to view in this way in view simply
        );                                                          $name variable to show data
    }

-----------------------------------------------------------------------------------------------

Summary of Steps:

1 Configure .env for SMTP settings.
2 Create a mailable class.
3 Create an email Blade view.
4 Send the email using the Mail facade.
```

Send Email Width Attachment

1 **php artisan make:controller EmailController**

2 **php artisan make:mail Welcomemail**

3 .env

```
MAIL_MAILER=smtp                          //smtp for gmail
MAIL_HOST=smtp.gmail.com                  //smtp.gmail.com for gmail
MAIL_PORT=587                             //587 port number for gmail
MAIL_USERNAME=anjeshravi8@gmail.com       //your username or mail id
MAIL_PASSWORD=fisvyknpxdjgajjq            //gmail app password, generate on google security
MAIL_ENCRYPTION=tls                       //tls encryption type for gmail
MAIL_FROM_ADDRESS="anjeshravi8@gmail.com" //send from mail id
MAIL_FROM_NAME="${APP_NAME}"
```

4 web.php

```php
<?php

use App\Http\Controllers\EmailController; //include controller file
use Illuminate\Support\Facades\Route;

Route::get('/contact', [EmailController::class, 'ContactForm']); //call ContactForm method
Route::post('/send-email', [EmailController::class, 'sendContactEmail'])->name('sendmail');
```

5 resources\views\mail\welcome-email.blade.php

```
    <h3>Hello Admin.</h3>
    <p>Name : {{$request['name']}}</p> //show name key value of request array
    <p>Email : {{$request['email']}}</p> //show email key value of request array
    <p>Subject : {{$request['subject']}}</p> //show subject key value of request array
    <p>Message : {{$request['message']}}</p> //show message key value of request array

    {{-- if you want show image in mail message use super global variable $message & embed()
     method in embed() method give image apslute path --}}
    <img src="{{$message->embed('image_path')}}" alt="">
```

6 resources\views\contact-form.blade.php

```
<form action="{{route('sendmail')}}" method="POST" enctype="multipart/form-data">
        @csrf
        <label for="name">Name</label>
        <input type="text" name="name" id="name" placeholder="Your Name"> //name filed

        <label for="email">Email Address</label>
        <input type="email" name="email" id="email" placeholder="Your Email"> //email filed

        <label for="subject">Subject</label>
        <input type="text" name="subject" id="subject" placeholder="Subject"> //subject filed

        <label for="message">Message</label>
        <input type="text" name="message" id="message" placeholder="Message"> //message filed

        <label for="file">File</label>
        <input type="file" name="attachment" id="file"> //choose file filed

        <input type="submit" value="Submit"> //submit button for submit form data to sendmail
</form>                                                                              route
```

Send Email Width Attachment-II

7 App\Http\Controllers\EmailController.php

```php
<?php

namespace App\Http\Controllers;

use App\Mail\Welcomeemail; //include mailable class in this example Welcomemail class
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Mail;

class EmailController extends Controller
{

    public function ContactForm() //custom method to return contact form view
    {
        return view('contact-form');
    }

-----------------------------------------------------------------------------------------------

    public function sendContactEmail(Request $request) //custom method for send email
    {
        $request->validate([
            'name' => 'required',
            'email' => 'required|email',
            'subject' => 'required|min:5|max:100',
            'message' => 'required|min:5|max:255',
            'attachment' => 'required|mimes:pdf,doc,docx,xls,xlsx|max:2048',
        ]); //validate the form request data

-----------------------------------------------------------------------------------------------
        //make file name current time & get original file extension
        $fileName = time() . "." . $request->file('attachment')->extension();

        //extract from request array file using file method then move file to uploads folder
        $request->file('attachment')->move('uploads', $fileName);


       $adminEmail = 'anjeshravi8@yahoo.com'; //email id to send email

-----------------------------------------------------------------------------------------------

       //using mail class to send mail, using all method to pass all data of request array
       $response = Mail::to($adminEmail)->send(new Welcomeemail($request->all(), $fileName));

       $response = Mail::to(mail id)->send(new Welcomeemail(form data, file name));
-----------------------------------------------------------------------------------------------

        if ($response) {//if mail send successfully then return back page with message
            return back()->with('success', 'Thank you for contact us');
        } else {
            return back()->with('error', 'unable to submit form data. Please try again');
        } //if mail not send redirect to back page with error message
  }
}
```

7 App\Mail\Welcomemail.php

```php
<?php

namespace App\Mail;

use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Mail\Mailable;
use Illuminate\Mail\Mailables\Attachment;
use Illuminate\Mail\Mailables\Content;
use Illuminate\Mail\Mailables\Envelope;
use Illuminate\Queue\SerializesModels;

class Welcomeemail extends Mailable
{
    use Queueable, SerializesModels;

    public $request; //initialize variable name request for getting data from cotroller
    public $fileName; //initialize variable name filename for file name

-----------------------------------------------------------------------------------------

    public function __construct($request, $fileName)
    {
        $this->request = $request; //assign request array value to request variable
        $this->fileName = $fileName; //assign file name to filename variable
    }
-----------------------------------------------------------------------------------------

    public function envelope(): Envelope
    {
        return new Envelope(
            subject: 'Contact Form', //return static or dynamic subject of mail
        );
    }
-----------------------------------------------------------------------------------------

    public function content(): Content
    {
        return new Content(
            view: 'mail.welcom-mail', //return view file for mail message
        );
    }

-----------------------------------------------------------------------------------------

    public function attachments(): array
    {
        $attachment = []; //empty array for attachmen

        if ($this->fileName) {
            $attachment = [
                Attachment::fromPath(public_path('/uploads/' . $this->fileName))
            ]; //pus file path name in attachment array using fromPath()method
        }
        return $attachment; //return attachment file to mail class
    }
}
-----------------------------------------------------------------------------------------

Attachment::fromStorage('file_path') //if your file in storage folder then use this method

Attachment::fromStorageDisk('s3','file_path'); //if your file in any third party disk/online
```

API Sanctum (Route & Model)

1. php artisan install:api
2. php artisan make:controlller API/AuthController
3. php artisan make:model Post
4. php artisan make:controller API/PostController --api

5 api.php

```php
<?php

use App\Http\Controllers\API\AuthController; //import Auth Controller
use App\Http\Controllers\API\PostController; //import Post Controller
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route; //import Route Controller

Route::post('signup', [AuthController::class, 'signup']); //route for signup
Route::post('login', [AuthController::class, 'login']); //route for login


Route::middleware('auth:sanctum')->group(function () { //protect posts route using middleware
    Route::apiResource('posts', PostController::class);
    Route::get('logout', [AuthController::class, 'logout']); //protect logout route
});
```

6 App\Models\User.php

```php
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable; //import Authenticable class as User
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens; //import Has Api Tokens

class User extends Authenticatable
{
    use HasFactory, Notifiable, HasApiTokens; //use trait
    protected $fillable = ['name', 'email', 'password',]; //allow for insert & update data
    protected $hidden = ['password']; //protect password column
    protected function casts(): array //this casts method convert password in hashed
    {
        return [
            'password' => 'hashed', // [column name => casts methods]
        ];
    }
}
```

7 App\Models\Post.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    use HasFactory;

    protected $fillable = ['title', 'description', 'image']; //allow to insert & update data
}
```

API Sanctum (AuthController: signup, logout)

8 App\Http\Controllers\API\AuthController.php

```php
<?php
namespace App\Http\Controllers\API;
use App\Http\Controllers\Controller;
use App\Models\User;                          //import User Model file
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;          //import Auth Class
use Illuminate\Support\Facades\Validator;     //import Validator Class

class AuthController extends Controller
{
    public function signup(Request $request) //method to register New User
    {
        $validateUser = Validator::make(     //Validate incoming request data
            $request->all(),
            [
                'name' => 'required',
                'email' => 'required|email|unique:users,email',
                'password' => 'required',
            ]
        );

        if ($validateUser->fails()) {       //Check if validation fails
            return response()->json([
                'status' => false,
                'message' => 'Validation Error',
                'errors' => $validateUser->errors()->all()
            ], 401);
        }

        $user = User::create([              //insert request data (new user data)
            'name' => $request->name,
            'email' => $request->email,
            'password' => $request->password,
        ]);

        return response()->json([           //return success response
            'status' => true,
            'message' => 'User Created Successfully.',
            'user' => $user,
        ], 200);

    }
-------------------------------------------------------------------------------------------

    public function logout(Request $request) //Logout method for logging out the user
    {
        $user = $request->user();  //Get the authenticated user from the request
        $user->tokens()->delete(); //Delete all API tokens associated with the user


        // Optionally, you could delete a specific token by calling:
        // $user->tokens(API KEY NAME)->delete();


        // Return a JSON response indicating the user has logged out successfully
        return response()->json([
            'status' => true,
            'message' => 'You Logged Out Successfully.',
        ], 200);

    }
}
```

API Sanctum (AuthController: login)

8 App\Http\Controllers\API\AuthController.php

```php
    public function login(Request $request) //method for authenticate the Loging credential
    {
        $validateUser = Validator::make(    //Validate incoming request data
            $request->all(),
            [
                'email' => 'required|email',
                'password' => 'required',
            ]
        );


        if ($validateUser->fails()) {       //Check if validation fails
            return response()->json([
                'status' => false,
                'message' => 'Authentication Fails.',
                'errors' => $validateUser->errors()->all()
            ], 404);
        }

        //Authenticate the user using Auth::attempt() & return success response
        if (Auth::attempt(['email' => $request->email, 'password' => $request->password])) {
            return response()->json([
                'status' => true,
                'message' => 'User Login Successfully.',
                'token' => Auth::user()->createToken('API Token')->plainTextToken,
                'token_type' => 'bearer'
            ], 200);
        } else {
            return response()->json([      //return fails response if Authenticate fails
                'status' => false,
                'message' => 'Email & Password does not match.',
            ], 401);
        }
    }
```

9 App\Http\Controllers\API\PostController.php

```php
    public function destroy(string $id) //method for delete post data
    {
        $imagePath = Post::select('image')->where('id', $id)->get(); //get image path

        $filePath = public_path() . '/uploads/' . $imagePath[0]['image']; //image path
        if (file_exists($filePath)) {
            unlink($filePath);              //unlink image (destroy/delete)
        }

        $post = Post::where('id', $id)->delete(); //delete post data from database by post id

        return response()->json( //return success response indicate post deleted successful
            'status' => true,
            'message' => 'Post Delete Successfully.',
            'post' => $post
        ], 200);
    }
}
```

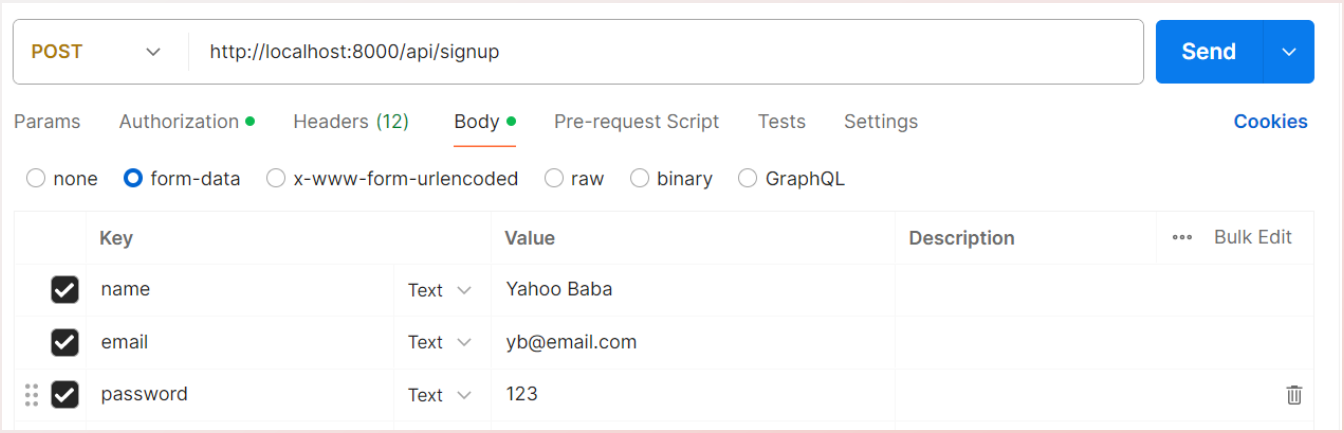
10 App\Http\Controllers\API\PostController.php

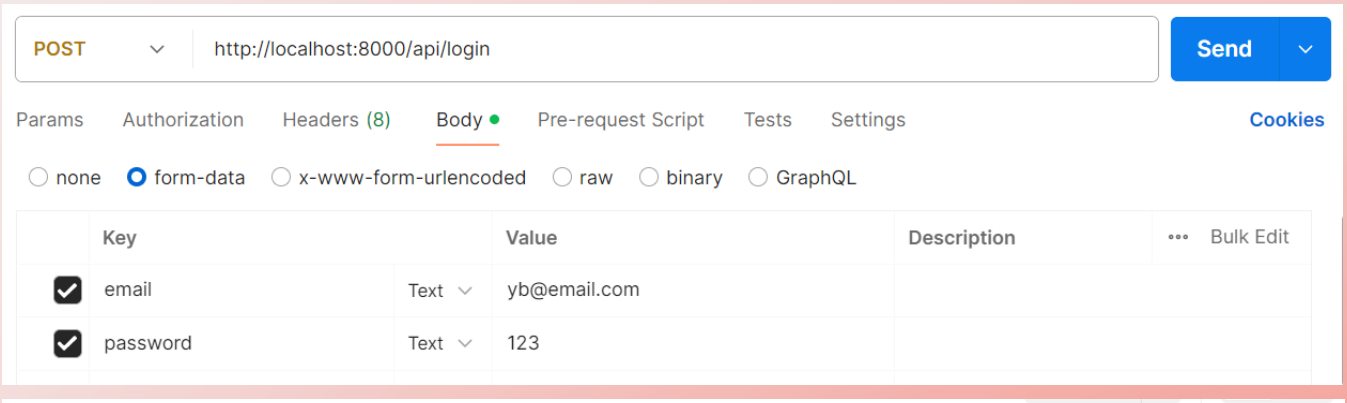
```php
<?php
namespace App\Http\Controllers\API;
use App\Models\Post;                        //import Post Model File
use Illuminate\Http\Request;                //import request class
use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\Validator;  //import validator class

class PostController extends Controller
{
    public function index()
    {
        $data['posts'] = Post::all();

        return response()->json([
            'status' => true,
            'message' => 'All Post Data',
            'data' => $data,
        ], 200);

     }

-------------------------------------------------------------------------------------------

    public function store(Request $request)
    {
        $validatePost = Validator::make(
            $request->all(),
            [
                'title' => 'required',
                'description' => 'required',
                'image' => 'required|mimes:png,jpg,jpeg,gif',
            ]
        );

        if ($validatePost->fails()) {
            return response()->json([
                'status' => false,
                'message' => 'Validation Error',
                'errors' => $validatePost->errors()->all()
            ], 401);
        }

        $img = $request->image;
        $imageName = time() . '.' . $img->getClientOriginalExtension();
        $img->move(public_path() . '/uploads', $imageName);


        $post = Post::create([
            'title' => $request->title,
            'description' => $request->description,
            'image' => $imageName,
        ]);


        return response()->json([
            'status' => true,
            'message' => 'New Post Created Successfully.',
            'post' => $post
        ], 200);

    }
```

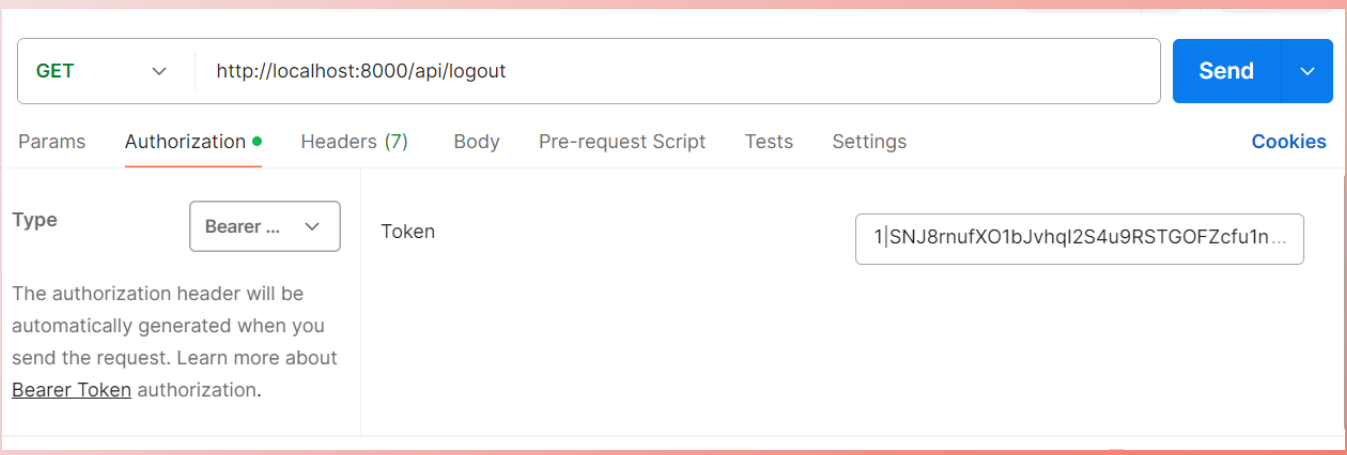
API Sanctum (PostController:show, update)

11 App\Http\Controllers\API\PostController.php

```php
    public function show(string $id)
    {
        $data['post'] = Post::findOrFail($id);

        return response()->json([
            'status' => true,
            'message' => 'Your Single Post.',
            'data' => $data
        ], 200);
    }
-------------------------------------------------------------------------------------------
    public function update(Request $request, string $id)
    {
        $validatePost = Validator::make(
            $request->all(),
            [
                'title' => 'required',
                'description' => 'required',
                'image' => 'required|mimes:png,jpg,jpeg,gif',
            ]
        );

        if ($validatePost->fails()) {
            return response()->json([
                'status' => false,
                'message' => 'Validation Error',
                'errors' => $validatePost->errors()->all()
            ], 401);
        }

        $postImage = Post::where('id', $id)->get('image');

               if ($request->file('image') != '') {

            $path = public_path() . '/uploads';
            if ($postImage[0]->image != '' && $postImage[0]->image != null) {
                $old_path = $path . $postImage[0]->image;
                if (file_exists($old_path)) {
                    unlink($old_path);
                }
            }
            $img = $request->image;
            $imageName = time() . '.' . $img->getClientOriginalExtension();
            $img->move(public_path() . '/uploads', $imageName);

        } else {
            $imageName = $postImage->image;
        }


        $post = Post::where(['id' => $id])->update([
            'title' => $request->title,
            'description' => $request->description,
            'image' => $imageName,
        ]);

        return response()->json([
            'status' => true,
            'message' => 'Post Update Successfully.',
            'post' => $post
        ], 200);
    }
```







Signup Endpoint
(POST /api/signup): Sends user details (name, email, and password) as form-data.
Used for registering a new user.

Login Endpoint
(POST /api/login):Sends credentials (email and password) as form-data.
Used to log in the user and retrieve a token.

Logout Endpoint
(GET /api/logout):Requires a Bearer Token in the Authorization header.
Logs out the user by invalidating the token